

Link Presentación:

https://www.canva.com/design/DAGktqDHwOU/JhQHngOXalZv-IY4eYWlwQ/edit?utm_content=DAGktqDHwOU&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton

Título	Endurecimiento avanzado en sistemas GNU/Linux y Windows: SELinux, AppLocker
---------------	---

Índice:

1.	meta
2.	Descripción
3.	Medios a utilizar
4.	Ejecución de GNU/Linux
5.	Ejecución de Windows
6.	Bibliografía/Webgrafía

1. meta

El objetivo principal de este proyecto es diseñar, implementar y probar un sistema de hardening avanzado tanto en entornos Linux como Windows, con el fin de reforzar la seguridad ante posibles amenazas. Garantizaremos la protección de servicios críticos, como SSH, configurando políticas restrictivas y utilizando SELinux, y estableceremos controles de ejecución de aplicaciones en Windows mediante AppLocker.

El proyecto busca demostrar la capacidad de:

- Aplicar técnicas de fortificación en sistemas operativos reales.
- Utilice herramientas específicas como SELinux y también auditd, Fail2Ban, nftables. En Windows, AppLocker.
- Crear políticas de seguridad personalizadas.
- Validar configuraciones mediante pruebas prácticas controladas.

Todo ello encaminado a conseguir un sistema robusto, con acceso seguro, minimizando la superficie de ataque y garantizando la trazabilidad de las acciones de los usuarios.

2. Descripción

El proyecto se estructura en dos partes principales:

- Hardening avanzado en Linux (Debian):

Se realizó una fortificación profunda del servicio SSH, modificando su configuración por defecto y aplicando medidas de control de acceso, detección de intrusiones y refuerzo de permisos de archivos. Además, se han implementado políticas SELinux para restringir y auditar el comportamiento de usuarios y procesos críticos. La protección se completa con un firewall basado en nftables que controla el tráfico entrante y saliente, así como medidas adicionales como el uso de Port Knocking y 2FA.

- Control de ejecución en Windows Server mediante AppLocker:

Se ha configurado un dominio de Active Directory para la gestión centralizada de políticas de seguridad. Las reglas de AppLocker se crearon para restringir la ejecución de programas, scripts e instaladores, garantizando que solo las aplicaciones autorizadas puedan ejecutarse en las computadoras cliente. El control se aplicó utilizando GPO específicos y se probó en una máquina cliente de Windows 10 unida a un dominio.

Cada cambio se realizó documentando los procedimientos, estableciendo pruebas específicas para validar su efectividad y asegurando que todos los mecanismos de seguridad estuvieran activos y operativos.

3. Medios a utilizar

1. MÁQUINAS VIRTUALES

- **Servidor Debian (Linux):**
 - Sistema operativo Debian 12.
 - Función principal: Servidor SSH fortificado mediante configuración manual y políticas de SELinux.
- **Cliente Kali Linux:**

- Sistema operativo Kali Linux actualizado.
- Función principal: Motor de ataque para realizar pruebas de acceso, escaneo de puertos y validación de medidas de endurecimiento aplicadas en el servidor Debian.
- **Servidor Windows Server 2019:**
 - Sistema operativo Windows Server 2019.
 - Funciones principales:
 - Controlador de dominio Active Directory.
 - Configure y administre las políticas de AppLocker para controlar la ejecución de aplicaciones en las computadoras cliente.
- **Clientes de Windows 10 Education:**
 - Sistema operativo Windows 10 Education.
 - Función principal: Computadoras unidas al dominio para aplicar y probar las políticas de restricción de AppLocker.

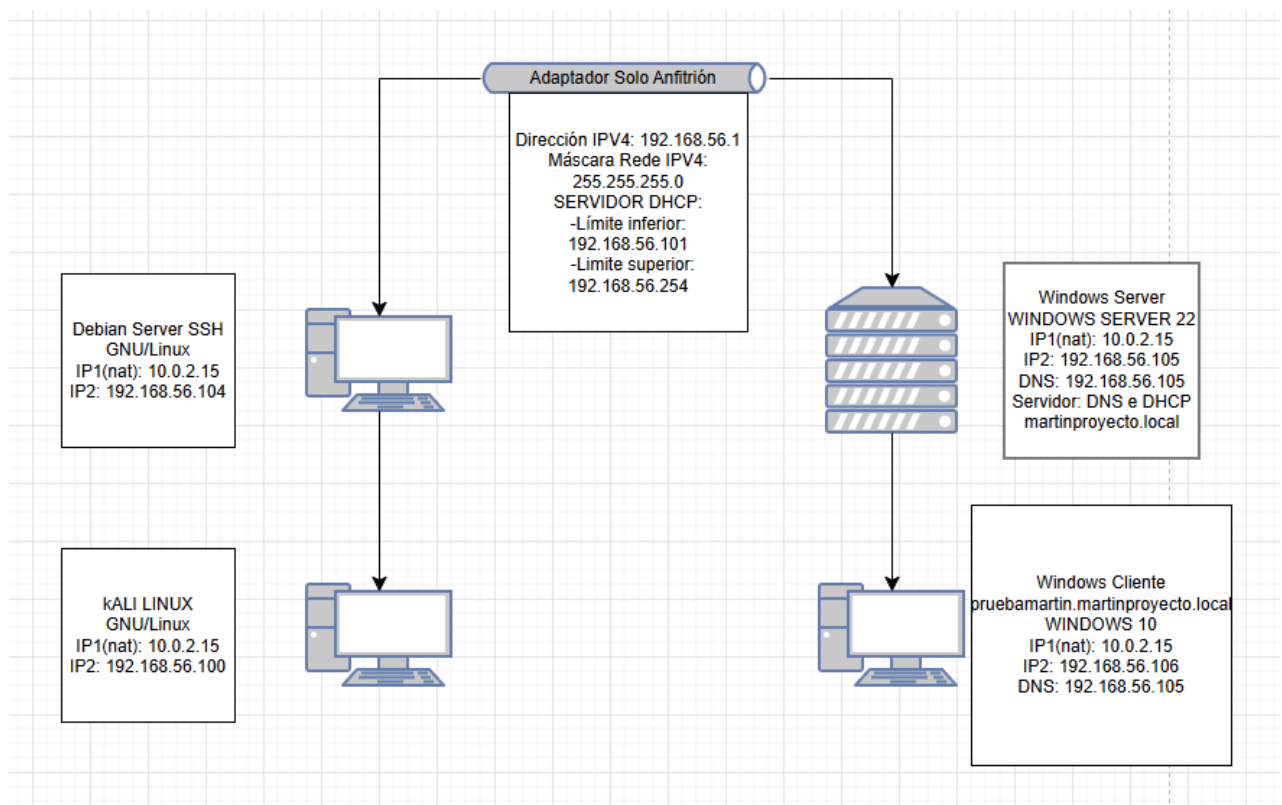
Cada máquina virtual ha sido configurada con una red adaptada a las necesidades específicas del proyecto: comunicación interna (Host Only) para el tráfico entre máquinas, y NAT para el acceso puntual a Internet (si es necesario)

Pero podemos tener en cuenta las características de cada máquina virtual utilizada. He creado una tabla para que puedas ver todo reflejado:

MÁQUINAS VIRTUALES				
	Servidor de Windows	Servidor Debian	Cliente Win	Cliente Kali
Sistema operativo	Servidor Windows 19	Debian 12	Ventanas 10	Kali Linux
Procesador y RAM	UPC: 1 RAM:4es	UPC:2 RAM:4es	UPC:1 RAM:2 GB	UPC:3 RAM:6es
Video	Memoria:128 MB Controlador:VBOX SVGA	Memoria:128 MB Controlador:Máquina virtual SVGA	Memoria:128 MB Controlador:VBOX SVGA	Memoria:128 MB Controlador:a VBOX

Disco Duro	Formato: VDI Tamaño: 200 GB Reserva espacio: Dinámico	Formato: VDI Tamaño: 100 GB Reserva espacio: Dinámico	Formato: VDI Tamaño: 100 GB Reserva espacio: Dinámico	Formato: VDI Tamaño: 120 GB Reserva espacio: Dinámico
-------------------	--	--	--	--

2. DIAGRAMA DE RED

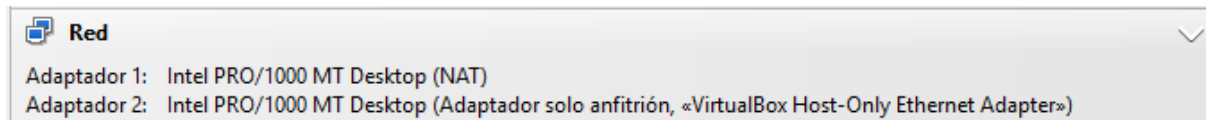


4. Ejecución de GNU/LINUX

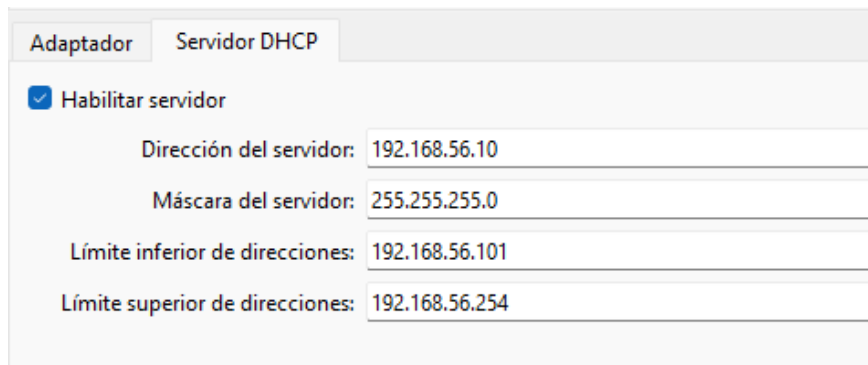
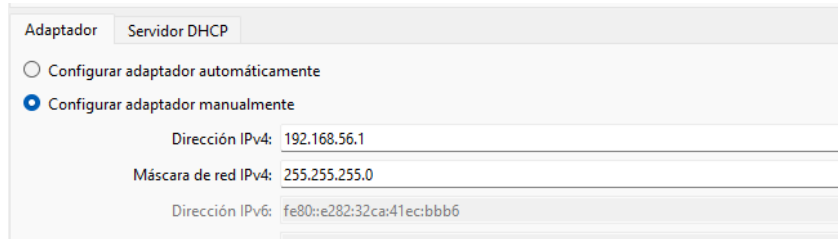
3. CREACIÓN DAS MÁQUINAS VIRTUALES

CONFIGURACIÓN DE RED VIRTUAL BOX

→ sería así para as duas maquinas

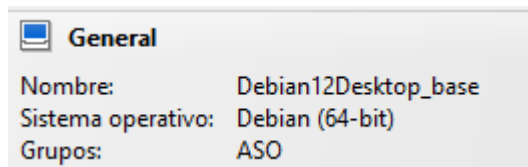


→ configuración sólo anfitrión

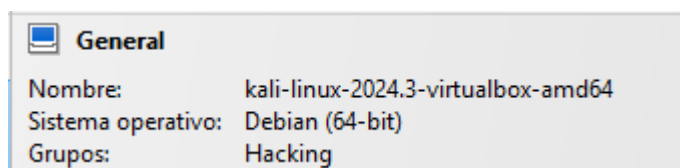


→ para configurar una ip automáticamente en ese rango
Añadimos e creamos as máquinas virtuales:

1. Escritorio Debian 12



2. Cliente atacante de Kali



4. CONFIGURACIÓN DE RED DE MÁQUINAS

CONFIGURACIÓN DE REDE DO DEBIAN

```

aso@base:~$ cat /etc/network/interfaces
# This file describes the network interfaces available on your system
# and how to activate them. For more information, see interfaces(5).

source /etc/network/interfaces.d/*

auto enp0s8
iface amp0s8 inet dhcp

# The loopback network interface
auto lo
iface lo inet loopback

```

ip a

```

aso@base:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:c6:24:cc brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic enp0s3
        valid_lft 86397sec preferred_lft 86397sec
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:ff:99:87 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.104/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s8
        valid_lft 505sec preferred_lft 505sec
    inet6 fe80::a00:27ff:feff:9987/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

```

→ probamos ping a internet:

```

root@base:/home/aso# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=255 time=17.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=255 time=15.7 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=255 time=15.5 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=255 time=16.0 ms
^C
--- 8.8.8.8 ping statistics ---
5 packets transmitted, 4 received, 20% packet loss, time 4131ms
rtt min/avg/max/mdev = 15.465/16.147/17.437/0.769 ms

```

→ Intentemos hacer ping a KALI Linux

```
aso@base:~$ ping 192.168.56.100
PING 192.168.56.100 (192.168.56.100) 56(84) bytes of data.
64 bytes from 192.168.56.100: icmp_seq=1 ttl=64 time=0.648 ms
64 bytes from 192.168.56.100: icmp_seq=2 ttl=64 time=0.600 ms
64 bytes from 192.168.56.100: icmp_seq=3 ttl=64 time=0.643 ms
64 bytes from 192.168.56.100: icmp_seq=4 ttl=64 time=0.748 ms
^C
--- 192.168.56.100 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3075ms
rtt min/avg/max/mdev = 0.600/0.659/0.748/0.054 ms
aso@base:~$
```

CONFIGURACIÓN DE REDE DO KALI ATACANTE

```
(kali㉿kali)-[~/ssh]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:ad:25:87 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
        valid_lft 60910sec preferred_lft 60910sec
    inet6 fd00::7c2f:a27c:a17:79dd/64 scope global dynamic noprefixroute
        valid_lft 84852sec preferred_lft 12852sec
    inet6 fe80::7a76:133b:68d0:6e86/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:ac:4e:32 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.100/24 brd 192.168.56.255 scope global noprefixroute eth1
        valid_lft forever preferred_lft forever
    inet 192.168.56.101/24 brd 192.168.56.255 scope global secondary dynamic noprefixroute eth1
        valid_lft 588sec preferred_lft 588sec
    inet6 fe80::ef0:3603:8e5f:291d/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

IP NAT: 10.0.2.15

Host solo IP: 192.168.56.100 / 192.168.56.101

→ Hacer ping a Debian:

```
(kali㉿kali)-[~/ssh]
└─$ ping 192.168.56.104
PING 192.168.56.104 (192.168.56.104) 56(84) bytes of data.
64 bytes from 192.168.56.104: icmp_seq=1 ttl=64 time=0.886 ms
64 bytes from 192.168.56.104: icmp_seq=2 ttl=64 time=0.810 ms
64 bytes from 192.168.56.104: icmp_seq=3 ttl=64 time=1.17 ms
64 bytes from 192.168.56.104: icmp_seq=4 ttl=64 time=0.620 ms
64 bytes from 192.168.56.104: icmp_seq=5 ttl=64 time=0.708 ms
^C
--- 192.168.56.104 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4053ms
rtt min/avg/max/mdev = 0.620/0.839/1.171/0.188 ms
```


→ Como indica o final do anterior comando, é necesario reiniciar para que se produza o proceso de etiquetado

```
/dev/sda1: clean, 211478/1831424 files, 1899968/7323904 blocks
[ OK ] Finished plymouth-read-write.service - Tell Plymouth To Write Out Runtime Data.
Mounting proc-sys-fs-binfmt_misc.mount - Arbitrary Executable File Formats File System...
[ OK ] Mounted proc-sys-fs-binfmt_misc.mount - Arbitrary Executable File Formats File System.
[ OK ] Finished systemd-binfmt.service - Set Up Additional Binary Formats.
[ OK ] Finished systemd-tmpfiles-setup.service - Create System Files and Directories.
Starting systemd-timesyncd.service - Network Time Synchronization...
Starting systemd-update-utmp.service - Record System Boot/Shutdown in UTMP...
[ OK ] Finished systemd-update-utmp.service - Record System Boot/Shutdown in UTMP.
[ OK ] Started systemd-timesyncd.service - Network Time Synchronization.
[ OK ] Reached target sysinit.target - System Initialization.
[ OK ] Reached target time-set.target - System Time Set.
Starting selinux-autorelabel.service - Relabel all filesystems...
Starting alsa-restore.service - Save/Restore Sound Card State...
[ OK ] Finished alsa-restore.service - Save/Restore Sound Card State.
[ OK ] Reached target sound.target - Sound Card.

*** Warning -- SELinux default policy relabel is required.
*** Relabeling could take a very long time, depending on file
*** system size and speed of hard drives.
libsemanage.add_user: user sddm not in password file
Warning: Skipping the following R/O filesystems:
/run/credentials/systemd-sysctl.service
/run/credentials/systemd-sysusers.service
/run/credentials/systemd-tmpfiles-setup-dev.service
/run/credentials/systemd-tmpfiles-setup.service
Relabeling / /home
42.5%
```

→ Una vez reiniciado y finalizado, comprobamos el estado y vemos que ya está activado:

```
aso@base:~$ sestatus
SELinux status:                enabled
SELinuxfs mount:              /sys/fs/selinux
SELinux root directory:       /etc/selinux
Loaded policy name:            default
Current mode:                  permissive
Mode from config file:         permissive
Policy MLS status:             enabled
Policy deny_unknown status:    allowed
Memory protection checking:    actual (secure)
Max kernel policy version:     33
```

→ Para continuar necesitamos cambiar el modo de operación a Enforcing, ya que el modo permisivo como se refleja en la captura anterior no bloquea ninguna acción.

```
aso@base:~$ sudo setenforce enforcing
[sudo] password for aso:
```

→ Aquí en este paso tuve un problema, ya que cada vez que ejecutaba el comando anterior, la máquina virtual fallaba.

→ Revisando los procesos están siendo bloqueados en permissive mode:

```
time->Fri Apr 4 00:27:49 2025
type=PROCTITLE msg=audit(1743719269.468:186): proctitle="/usr/bin/gnome-shell"
type=SYSCALL msg=audit(1743719269.468:186): arch=c000003e syscall=10 success=yes exit=0 a0=315380201000
a1=1000 a2=5 a3=7ffc19d4a080 items=0 ppid=1939 pid=2118 auid=1000 uid=1000 gid=1000 euid=1000 suid=1000
fsuid=1000 egid=1000 sgid=1000 fsgid=1000 tty=(none) ses=3 comm="gnome-shell" exe="/usr/bin/gnome-shell"
subj=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 key=(null)
type=AVC msg=audit(1743719269.468:186): avc: denied { execmem } for pid=2118 comm="gnome-shell" scontext=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 tcontext=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023 tclass=process permissive=1
```

→ Podemos concluir que SELinux está bloqueando la ejecución de memoria (execmem) para GNOME Shell. Esto puede provocar que la máquina se congele al cambiar al modo de aplicación.

→ Ahora permitiremos específicamente que gnome-shell use execmem sin deshabilitar SELinux:

```
aso@base:~$ sudo ausearch -c "gnome-shell" --raw | audit2allow -M gnome_shell_execmem
***** IMPORTANT *****
To make this policy package active, execute:

semodule -i gnome_shell_execmem.pp
```

→ Con esto busca en los registros restricciones en gnome-shell y crea el módulo para permitir la acción

→ Luego nos pide que ejecutamos sudo semodule -i gnome shell_exec mem.pp para instalar el módulo de política creado

```
aso@base:~$ sudo semodule -i gnome_shell_execmem.pp
libsemanage.add_user: user sddm not in password file
```

→ Lo ejecutamos y nos dice que la política se está ejecutando en un usuario que no existe en el sistema. Intentemos crear el usuario temporalmente para que SELinux no dé un error:

```
aso@base:~$ sudo useradd -r -s /sbin/nologin sddm
```

→ Lo creamos y ahora intentemos instalar la política nuevamente

```
aso@base:~$ sudo semodule -i gnome_shell_execmem.pp
```

→ Ahora ya no nos da el error y comprobaremos que el módulo está cargado correctamente:

```
aso@base:~$ sudo semodule -l | grep gnome_shell_execmem
gnome_shell_execmem
```

→ Una vez que hayamos aplicado exitosamente el módulo, intentaremos nuevamente cambiar SELinux a enforcing sin ningún error.

```
aso@base:~$ sudo setenforce enforcing
[sudo] password for aso:
```

→ Como expliqué antes, podemos hacer que la aplicación sea persistente en todos los reinicios:

```
GNU nano 7.2 /etc/selinux/config *
# This file controls the state of SELinux on the system.
# SELINUX= can take one of these three values:
# enforcing - SELinux security policy is enforced.
# permissive - SELinux prints warnings instead of enforcing.
# disabled - No SELinux policy is loaded.
SELINUX=enforcing
# SELINUXTYPE= can take one of these two values:
# default - equivalent to the old strict and targeted policies
# mls      - Multi-Level Security (for military and educational use)
# src      - Custom policy built from source
SELINUXTYPE=default

# SETLOCALDEFS= Check local definition changes
SETLOCALDEFS=0
```

→ Ahora guardamos y reiniciamos nuevamente para verificar que el modo no cambie

```
aso@base:~$ sestatus
SELinux status:                enabled
SELinuxfs mount:              /sys/fs/selinux
SELinux root directory:       /etc/selinux
Loaded policy name:            default
Current mode:                  enforcing
Mode from config file:         enforcing
Policy MLS status:             enabled
Policy deny_unknown status:    allowed
Memory protection checking:    actual (secure)
Max kernel policy version:     33
aso@base:~$
```

→ Como podemos ver, tenemos un modo de aplicación persistente.

6. FORTIFICACIÓN DE SSH CON SELINUX

→ FORTIFICACIÓN:

1. Desactivamos el acceso root por SSH.
2. Reforzamos el uso de claves SSH en lugar de contraseñas.
3. Cambiamos el puerto a un número alto y lo registramos en SELinux.
4. Limitamos a los usuarios que pueden iniciar sesión.
5. Configuramos un firewall que sólo permite conexiones SSH seguras.
6. Engañamos a Fail2Ban para bloquear intentos de acceso sospechosos.
7. Habilitamos SELinux MLS para un control más estricto.
8. Monitoreamos eventos en SSH con SELinux y creamos políticas personalizadas.
9. Implementamos 2FA para agregar una capa adicional de seguridad.
10. Implemente Port Knocking para ocultar el puerto SSH.
11. Establecer umask para restringir permisos por defecto.
12. Configure SSH Escape Timing para mitigar ataques de fuerza bruta.
13. Restringir la ejecución de comandos a través de SSH
14. Utilice SELinux para hacer chroot a los usuarios SSH
15. Auditoría de comandos con auditd (SELinux-aware)
16. Bloquear conexiones TCP salientes con nftables (Firewall de salida)

Este nivel de fortificación convierte a SSH en una **clase inexpugnable** En un sistema Linux tenemos una política SELinux avanzada.

1. Restringir o acceso root e usar autenticación por clave pública SSH

→ Accedemos o archivo de configuración de SSH:

/etc/ssh/sshd_config

```
aso@base:~$ sudo nano /etc/ssh/sshd_config
[sudo] password for aso:
```

→ Ahora modificamos las opciones:

```
#LoginGraceTime 2m
PermitRootLogin no
#StrictModes yes
#MaxAuthTries 6
#MaxSessions 10

PubkeyAuthentication yes

# Expect .ssh/authorized_keys2
#AuthorizedKeysFile .ssh/authorized_keys
# To disable tunneled clear text
PasswordAuthentication no
#PermitEmptyPasswords no
```

→ Con esta nueva configuración prohibimos el acceso a root y por contraseña e permitirlo por clave pública.

```
aso@base:~$ sudo systemctl restart sshd
```

→ Reiniciamos el servicio SSH para aplicar los cambios

2. Cambie el puerto SSH y asegúrelo con SELinux

Para evitar ataques automatizados, cambiaremos el puerto y configuraremos SELinux para permitirlo.

→ Volvemos a acceder al archivo de configuración de SSH:

```
aso@base:~$ sudo nano /etc/ssh/sshd_config
```

```
Include /etc/ssh/sshd_config.d/
```

```
#Port 22
```

```
#AddressFamily any
```

```
#ListenAddress 0.0.0.0
```

```
#ListenAddress ::
```

→ Vamos a cambiar el puerto SSH predeterminado, 22, a un nuevo puerto, 2222:

```
Include /etc/ssh/sshd_config.d/*.conf
```

```
Port 2222
```

```
#AddressFamily any
```

```
#ListenAddress 0.0.0.0
```

```
#ListenAddress ::
```

```
#HostKey /etc/ssh/ssh_host_rsa_key
```

```
#HostKey /etc/ssh/ssh_host_ecdsa_key
```

→ Ahora tenemos que configurar SELinux para permitir este puerto

```
aso@base:~$ sudo semanage port -a -t ssh_port_t -p tcp 2222
```

→ Verificamos:

```
aso@base:~$ sudo semanage port -l | grep ssh
```

```
ssh_port_t                                tcp      2222, 22
```

→ Bloquearemos el puerto 22 para que solo funcione con el puerto 2222:

```
aso@base:~$ sudo semanage port -d -t ssh_port_t -p tcp 22
```

```
ValueError: Port tcp/22 is defined in policy, cannot be deleted
```

→ SELinux no permite eliminar el puerto 22 porque está definido en la política predeterminada. Bloqueemos las conexiones entrantes a través de dicho puerto con

Nftables:

```
aso@base:~$ sudo systemctl status nftables
● nftables.service - nftables
   Loaded: loaded (/lib/systemd/system/nftables.service; enabled; preset:
   Active: active (exited) since Mon 2025-04-07 12:36:54 CEST; 4s ago
     Docs: man:nft(8)
           http://wiki.nftables.org
   Process: 3872 ExecStart=/usr/sbin/nft -f /etc/nftables.conf (code=exitec
   Main PID: 3872 (code=exited, status=0/SUCCESS)
      CPU: 7ms
```

→ Confirmamos que Nftables está activado, ahora imos crear la tabla filter:

```
aso@base:~$ sudo nft add table inet filter
aso@base:~$ sudo nft add chain inet filter input { type filter hook input priority 0 \; }
```

→ Ahora bloqueamos o puerto 22:

```
aso@base:~$ sudo nft add rule inet filter input tcp dport 22 drop
```

→ Lo hacemos persistente después de los reinicios:

```
aso@base:~$ sudo sh -c "nft list ruleset > /etc/nftables.conf"
```

```
aso@base:~$ sudo nft list ruleset
table inet filter {
    chain input {
        type filter hook input priority filter; policy accept;
        tcp dport 22 drop
    }

    chain forward {
        type filter hook forward priority filter; policy accept;
    }

    chain output {
        type filter hook output priority filter; policy accept;
    }
}
```

→ Confirmamos que está creada la regla que bloquea o porta 22.

3. Restringir o acceso SSH a usuarios específicos

→ Para que sólo ciertos usuarios puedan conectarse vía SSH:

```
aso@base:~$ sudo nano /etc/ssh/sshd_config
```

→ Añadimos a opción:

```
AllowUsers aso martin martin1 martin2
```

→ Una vez más reiniciamos el servicio

4. Proteger SSH con Firewall (UFW y NFTABLES)

→ Opción 1, utilizando NFTABLES

Añadimos a regla de permitir o puerto 2222

```
aso@base:~$ sudo nft add rule inet filter input tcp dport 2222 accept
```

Comprobamos:

```
aso@base:~$ sudo nft list ruleset
table inet filter {
    chain input {
        type filter hook input priority filter;
        tcp dport 22 drop
        tcp dport 2222 accept
    }

    chain forward {
        type filter hook forward priority filte
    }
}
```

→ Opción 2, utilizando UFW

```
aso@base:~$ sudo ufw allow 2222/tcp
Rules updated
Rules updated (v6)
```

Habilitamos el nuevo puerto 2222

Comprobamos que se realizaron las modificaciones:

```
aso@base:~$ sudo ufw status
Status: active
```

To	Action	From
--	-----	----
2222/tcp	ALLOW	Anywhere
2222/tcp (v6)	ALLOW	Anywhere (v6)

Ahora imos bloquear o porto 22 con UFW:

```
aso@base:~$ sudo ufw deny 22/tcp
Rule added
Rule added (v6)
```

Verificamos:

```
aso@base:~$ sudo ufw status
Status: active
```

To	Action	From
--	-----	----
2222/tcp	ALLOW	Anywhere
22/tcp	DENY	Anywhere
2222/tcp (v6)	ALLOW	Anywhere (v6)
22/tcp (v6)	DENY	Anywhere (v6)

5. Configurar límites de intentos de conexión con Fail2Ban

Esta medida se centra principalmente en prevenir ataques de fuerza bruta.

→ Instalamos Fail2Ban:

```
aso@base:~$ sudo apt install fail2ban -y
Lendo as listas de paquetes... Feito
```

→ Creamos unha configuración específica de SSH en Fail2Ban, para eso tenemos que crear o seguinte arquivo:

```
aso@base:~$ sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
```

→ Una vez creado modifícalo:

→ Añadimos las siguientes reglas:

```
[sshd]

# To use more aggressive sshd modes :
# normal (default), ddos, extra or a
# See "tests/files/logs/sshd" or "fi
#mode    = normal
enabled = true
port = ssh
maxretry = 3
bantime = 600
findtime = 600
logpath = %(sshd_log)s
backend = %(sshd_backend)s
```

→ Establecemos el número máximo de intentos fallidos en 3 y estableceremos un bloqueo de 600 segundos.

→ Comprobamos o estado:

```
aso@base:~$ sudo systemctl status fail2ban
● fail2ban.service - Fail2Ban Service
   Loaded: loaded (/lib/systemd/system/fail2ban.se
   Active: active (running) since Tue 2025-04-08 0
     Docs: man:fail2ban(1)
  Main PID: 7905 (fail2ban-server)
    Tasks: 5 (limit: 4620)
   Memory: 14.3M
      CPU: 222ms
   CGroup: /system.slice/fail2ban.service
           └─7905 /usr/bin/python3 /usr/bin/fail2b
```

```
Abr 08 00:10:20 base systemd[1]: Started fail2ban.se
Abr 08 00:10:21 base fail2ban-server[7905]: 2025-04-
```

→ Verifiquemos también que Fail2Ban esté protegiendo el servicio SSH:

```
aso@base:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed:    0
| `-- File list:      /var/log/auth.log
`-- Actions
   |- Currently banned: 0
   |- Total banned:    0
   `-- Banned IP list:
```

→ Como vemos, xa está funcionando, por ahora, non hai ningún intento fallido nin IPs baneadas

6. Habilitar el modo SELinux MLS para SSH

El Control de Acceso Obligatorio (MLS) de SELinux es una opción avanzada que le permite establecer políticas de seguridad más estrictas y granulares, controlando el acceso a los recursos del sistema en función del nivel de seguridad asociado a cada proceso y objeto. Al

habilitar este modo para SSH, podemos fortalecer la seguridad de las conexiones SSH, garantizando que solo los usuarios autorizados puedan acceder a sistemas sensibles.

→ Comprobamos el archivo de configuración de SELinux:

```
SELINUX=enforcing
# SELINUXTYPE= can take one of these t
# default - equivalent to the old stri
# mls      - Multi-Level Security (for
# src      - Custom policy built from s
SELINUXTYPE=default
```

→ Lo cambiamos por poner:

```
# default - equivalent to
# mls      - Multi-Level S
# src      - Custom policy
SELINUXTYPE=mls
```

→ Comprobemos que realmente está activo:

```
aso@base:~$ sudo sestatus | grep "Policy MLS status"
[sudo] password for aso:
Policy MLS status:                enabled
```

7. Habilite SELinux para monitorear y proteger SSH mediante el establecimiento de reglas

→ Activamos a primera regla de auditoría de SSH:

```
aso@base:~$ sudo auditctl -w /etc/ssh/sshd_config -p wa -k ssh_audit
```

→ Ahora podemos ver los registros de SELinux relacionados con SSH:

`sudo ausearch -m AVC,USUARIO AVC -unidad SSD`

```
aso@base:~$ sudo ausearch -m AVC,USER_AVC -c sshd
----
time->Fri Apr 4 01:01:43 2025
type=PROCTITLE msg=audit(1743721303.643:162): proctitle=2F7573722F7362696E2F73736864002D74
type=SYSCALL msg=audit(1743721303.643:162): arch=c000003e syscall=138 success=no exit=-13 a0=3 a1=7ffde0
03d990 a2=0 a3=0 items=0 ppid=1 pid=573 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0
fsgid=0 tty=(none) ses=4294967295 comm="sshd" exe="/usr/sbin/sshd" subj=system_u:system_r:sshd_t:s0 key=
(null)
type=AVC msg=audit(1743721303.643:162): avc: denied { getattr } for pid=573 comm="sshd" name="/" dev=
"proc" ino=1 scontext=system_u:system_r:sshd_t:s0 tcontext=system_u:object_r:proc_t:s0 tclass=filesystem
permissive=0
----
time->Fri Apr 4 01:01:43 2025
type=PROCTITLE msg=audit(1743721303.803:174): proctitle=2F7573722F7362696E2F73736864002D44
type=SYSCALL msg=audit(1743721303.803:174): arch=c000003e syscall=138 success=no exit=-13 a0=3 a1=7ffda0
0f2310 a2=0 a3=0 items=0 ppid=1 pid=590 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0
fsgid=0 tty=(none) ses=4294967295 comm="sshd" exe="/usr/sbin/sshd" subj=system_u:system_r:sshd_t:s0 key=
(null)
type=AVC msg=audit(1743721303.803:174): avc: denied { getattr } for pid=590 comm="sshd" name="/" dev=
"proc" ino=1 scontext=system_u:system_r:sshd_t:s0 tcontext=system_u:object_r:proc_t:s0 tclass=filesystem
permissive=0
```

Esta regla es útil para rastrear quién cambia la configuración de SSH y cuándo, pero no audita la actividad del servicio SSH en sí.

Para auditar los intentos de login por SSH (incluyendo los intentos fallidos como root), imod añadir reglas de auditoria que rastreen eventos relacionados con la autenticación y el proceso sshd.

→ Creación de las reglas:

sudo auditctl -w /var/log/secure -p wa -k ssh_login # Para sistemas que usan este log
sudo auditctl -w /var/log/auth.log -p wa -k ssh_login

Estos monitorean los archivos de registro de autenticación en busca de escrituras, donde generalmente se registran los intentos de inicio de sesión.

sudo auditctl -a always,exit -F path=/usr/sbin/sshd -F perm=x -k sshd_exec

Realizar un seguimiento de la ejecución del propio sshd

sudo auditctl -a siempre, salir -F arch=b64 -F 'auid>=1000' -F 'auid!=4294967295' -S connect -F path=/var/run/sshd.sock -k ssh_connections

o

sudo auditctl -a always,exit -F arch=b64 -S connect -F 'auid>=1000' -F 'auid!=4294967295' -F path=/var/run/sshd.sock -F key=ssh_connections

Realiza un seguimiento de las conexiones del socket sshd realizadas por usuarios reales.

sudo auditctl -a siempre, salir -F arch=b64 -S accept -F 'auid>=1000' -F 'auid!=4294967295' -F key=ssh_access

Se registrará cuando el servidor SSH acepte una nueva conexión de un usuario real en sistemas de 64 bits.

sudo auditctl -a siempre, salir -F arch=b64 -S fork -F 'auid>=1000' -F 'auid!=4294967295' -F key=ssh_access

Registrará la creación de nuevos procesos por usuarios reales no en contexto de SSH en sistemas de 64 bits.

sudo auditctl -a siempre, salir -F arch=b64 -S execve -F 'auid>=1000' -F 'auid!=4294967295' -F key=ssh_access

Registrará la ejecución de comandos por parte de usuarios reales dentro de sesiones SSH en sistemas de 64 bits.

→ Comprobamos que se crearon:

```
aso@base:~$ sudo auditctl -l
-w /var/log/secure -p wa -k ssh_login
-w /var/log/auth.log -p wa -k ssh_login
-w /usr/sbin/sshd -p x -k sshd_exec
-a always,exit -F arch=b64 -S connect -F auid>=1000 -F auid!=-1 -F path=/var/run/sshd.sock -F key=ssh_connections
-a always,exit -F arch=b64 -S accept -F auid>=1000 -F auid!=-1 -F key=ssh_access
-a always,exit -F arch=b64 -S fork -F auid>=1000 -F auid!=-1 -F key=ssh_access
-a always,exit -F arch=b64 -S execve -F auid>=1000 -F auid!=-1 -F key=ssh_access
```

8. Configurar autenticación de dos factores (2FA)

→ Para añadir outra capa de seguridade, instalamos google-authenticator:

```
aso@base:~$ sudo apt install libpam-google-authenticator -y
Lendo as listas de paquetes... Feito
Construindo a árbore de dependencias... Feito
Lendo a información do estado... Feito
O seguinte paquete foi instalado automaticamente e xa non é nec
```

→ Una vez instalado, configúrelo:

```
aso@base:~$ google-authenticator

Do you want authentication tokens to be time-based (y/n) y
Warning: pasting the following URL into your browser exposes the OTP secret to Google:
https://www.google.com/chart?chs=200x200&chld=M|0&cht=qr&chl=otpauth://totp/aso@base%3Fsecret%3DH2U76D
WIFYBOI5RYETV5XSTQ3DE%26issuer%3Dbase
```



```
Your new secret key is: H2U76DWFYBOI5RYETV5XSTQ3DE
Enter code from app (-1 to skip):
Code incorrect (correct code 017955). Try again.
Enter code from app (-1 to skip):
Code incorrect (correct code 017955). Try again.
Enter code from app (-1 to skip): 017955
Code confirmed
Your emergency scratch codes are:
67486440
53487906
31676802
79324645
52661403
```

```
Do you want me to update your "/home/aso/.google_authenticator" file? (y/n) y
```

```
Do you want to disallow multiple uses of the same authentication
token? This restricts you to one login about every 30s, but it increases
your chances to notice or even prevent man-in-the-middle attacks (y/n) y
```

```
By default, a new token is generated every 30 seconds by the mobile app.
In order to compensate for possible time-skew between the client and the server,
we allow an extra token before and after the current time. This allows for a
time skew of up to 30 seconds between authentication server and client. If you
experience problems with poor time synchronization, you can increase the window
from its default size of 3 permitted codes (one previous code, the current
code, the next code) to 17 permitted codes (the 8 previous codes, the current
code, and the 8 next codes). This will permit for a time skew of up to 4 minutes
between client and server.
```

```
Do you want to do so? (y/n) y
```

```
If the computer that you are logging into isn't hardened against brute-force
login attempts, you can enable rate-limiting for the authentication module.
By default, this limits attackers to no more than 3 login attempts every 30s.
Do you want to enable rate-limiting? (y/n) y
```

→ Una vez realizada la configuración inicial de Google Authenticator, activamos el 2FA en el archivo `/etc/pam.d/ssh`:

```
# to run in the user's context should be run aft
session [success=ok ignore=ignore module_unknowr

# Standard Un*x password updating.
@include common-password

auth required pam_google_authenticator.so
```

→ Una vez añadido el requisito de Google Authenticator, tenemos que habilitar dos opciones en el archivo `/etc/ssh/sshd_config`:

```
#PermitEmptyPasswords no

ChallengeResponseAuthentication yes
PAMAuthenticationViaKbdInt yes
```

Una vez terminado, reinicie el servicio ssh

9. Implementación de Port Knocking

Port Knocking oculta el puerto SSH, haciéndolo inaccesible hasta que se envíe una secuencia específica de paquetes

→ Instalamos Port Knocking

```
aso@base:~$ sudo apt install knockd
Lendo as listas de paquetes... Feito
Construindo a árvore de dependencias... Feito
Lendo a informação do estado... Feito
O seguinte paquete foi instalado automaticamei
libdbus-qlib-1-2
```

→ Una vez instalado, acceda al archivo de configuración `/etc/knockd.conf` e añadimos o novo porto:

```
aso@base:~$ sudo nano /etc/knockd.conf
```

```
GNU nano 7.2 /etc/knockd.conf *
[options]
    UseSyslog

[openSSH]
    sequence      = 7000,8000,9000
    seq_timeout   = 5
    command       = /sbin/iptables -A INPUT -s %IP% -p tcp --dport 2222 -j ACCEPT
    tcpflags      = syn

[closeSSH]
    sequence      = 9000,8000,7000
    seq_timeout   = 5
    command       = /sbin/iptables -D INPUT -s %IP% -p tcp --dport 2222 -j ACCEPT
    tcpflags      = syn
```

→ Como podemos ver, configuré una secuencia de puertos (7000,8000,9000) para abrir SSH

→ Activamos o servicio Knockd:

```
aso@base:~$ sudo systemctl enable knockd --now
Synchronizing state of knockd.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable knockd
Created symlink /etc/systemd/system/multi-user.target.wants/knockd.service → /lib/systemd/system/knockd.service.
```

→ Realizamos un status para comprobar o estado

```
aso@base:~$ sudo systemctl status knockd
× knockd.service - Port-Knock Daemon
   Loaded: loaded (/lib/systemd/system/knockd.service; enabled; preset: enabled)
   Active: failed (Result: exit-code) since Tue 2025-04-08 23:49:26 CEST; 1min 14s ago
     Duration: 49ms
    Docs: man:knockd(1)
   Process: 8518 ExecStart=/usr/sbin/knockd $KNOCKD_OPTS (code=exited, status=1/FAILURE)
    Main PID: 8518 (code=exited, status=1/FAILURE)
      CPU: 9ms

Abr 08 23:49:26 base systemd[1]: Started knockd.service - Port-Knock Daemon.
Abr 08 23:49:26 base knockd[8518]: could not open eth0: eth0: No such device exists (No su
Abr 08 23:49:26 base systemd[1]: knockd.service: Main process exited, code=exited, status=
Abr 08 23:49:26 base systemd[1]: knockd.service: Failed with result 'exit-code'.
lines 1-13/13 (END)
```

→ Vemos un error que indica que knockd no puede encontrar la interfaz de red eth0

→ Temos que acceder o archivo /etc/default/knockd, e añadir a interfaz de rede correcta:

```
GNU nano 7.2 /etc/default/knockd
# control if we start knockd at init or not
# 1 = start
# anything else = don't start
# PLEASE EDIT /etc/knockd.conf BEFORE ENABLING
START_KNOCKD=0

# command line options
KNOCKD_OPTS="-i enp0s8"
```

→ Ahora reiniciamos:

```
aso@base:~$ sudo systemctl restart knockd
```

→ Volvemos a comprobar o estado:

```
aso@base:~$ sudo systemctl status knockd
● knockd.service - Port-Knock Daemon
   Loaded: loaded (/lib/systemd/system/knockd.service; enabled; preset: enabled)
   Active: active (running) since Fri 2025-04-25 17:17:08 CEST; 3s ago
     Docs: man:knockd(1)
    Main PID: 17529 (knockd)
      Tasks: 1 (limit: 4620)
     Memory: 616.0K
        CPU: 20ms
    CGroup: /system.slice/knockd.service
           └─17529 /usr/sbin/knockd -i enp0s8

Abr 25 17:17:08 base systemd[1]: Started knockd.service - Port-Knock Daemon.
Abr 25 17:17:08 base knockd[17529]: starting up, listening on enp0s8
```

10. Configurar umask para restringir permisos

Esto es para evitar que los archivos creados por usuarios SSH tengan permisos inseguros.
→ Vou añadir umask 077 nos arquivos de configuración do sistema e de Bash para añadir os novos permisos, os cales son moi restrictivos. Solo propietario terá permisos de lectura e escritura

```
aso@base:~$ echo "umask 077" | sudo tee -a /etc/profile /etc/bash.bashrc
umask 077
```

→ Comprobemos que realmente se añadió a los archivos:

/etc/perfil

```
aso@base:~$ cat /etc/profile
# /etc/profile: system-wide .profile file for the Bourne shell (sh(1)
# and Bourne compatible shells (bash(1), ksh(1), ash(1), ...).

if [ "$(id -u)" -eq 0 ]; then
    PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
fi

if [ -d /etc/profile.d ]; then
    for i in /etc/profile.d/*.sh; do
        if [ -r $i ]; then
            . $i
        fi
    done
    unset i
fi
umask 077
```

/etc/bash.bashrc

```
aso@base:~$ cat /etc/bash.bashrc
# System-wide .bashrc file for interactive bash

# To enable the settings / commands in this file
# this file has to be sourced in /etc/profile.

# ...
return 127
fi
}
fi
umask 077
```

→ Ahora vamos a guardar los cambios realizados en los dos archivos:

```
aso@base:~$ source /etc/profile
aso@base:~$ source /etc/bash.bashrc
```

→ Verificamos:

```
aso@base:~$ umask
0077
```

11. Configurar el tiempo de escape de SSH

El objetivo es aumentar la seguridad del servidor SSH desconectando automáticamente las sesiones que han estado activas durante un tiempo, para que no puedan ser explotadas.

→ Tenemos que acceder al archivo de configuración del servidor SSH /etc/ssh/sshd_config:

```
aso@base:~$ sudo nano /etc/ssh/sshd_config
```

→ Una vez dentro, modificamos las siguientes dos opciones:

```
#PermitTTY yes
PrintMotd no
#PrintLastLog yes
#TCPKeepAlive yes
#PermitUserEnvironment no
#Compression delayed
ClientAliveInterval 300
ClientAliveCountMax 2
#UseDNS no
#PidFile /run/sshd.pid
```

→ El servidor enviará un mensaje cada 300 segundos, si el cliente no responde se asume que se ha perdido la conexión. Además, si el servidor envía dos mensajes "keepalive" consecutivos y no recibe respuesta, la conexión también se terminará.

→ Reiniciamos ssh para guardar los cambios

```
aso@base:~$ sudo systemctl restart ssh
```


12. Restringir la ejecución de comandos a través de SSH (script personalizado + SELINUX: MLS y RBAC)

PARTE 1

PAGLimitemos los comandos que un usuario puede ejecutar al iniciar sesión a través de SSH. Esto es útil para usuarios con permisos restringidos, ya que les impide explorar el sistema o ejecutar comandos peligrosos.

-Los usuarios que se conecten a través de SSH solo podrán ejecutar comandos de lectura (como ls, cat, less, etc.).

-No podrán modificar archivos, utilizar nano, rm, mkdir, sudo, ni subir archivos.

→ Primero, vamos a crear un grupo de usuarios restringido

```
aso@base:~$ sudo groupadd ssh_readonly
```

→ Asignar los usuarios SSH (que queramos) a este grupo

```
aso@base:~$ sudo usermod -aG ssh_readonly martin
```

Se queremos añadir otros usuarios como por ejemplo Aso, teremos que cambiar martin polo nombre do usuario.

→ Vamos a crear un shell personalizado para la restricción de comandos:

```
aso@base:~$ sudo nano /usr/local/bin/ssh_readonly_shell.sh
```

```
GNU nano 7.2 /usr/local/bin/ssh_readonly_shell.sh *
#!/bin/bash

# Lista blanca de comandos permitidos
allowed_cmds="ls|cat|pwd|whoami|uptime|id|date|hostname|df|free|top|less|more|head|tail|uname|env|who|w|man|cal|which"

# Extraemos o comando que intenta usar o usuario
user_cmd=$(echo "$SSH_ORIGINAL_COMMAND" | awk '{print $1}')

# Comprobamos se o comando está na lista blanca
if [[ "$user_cmd" =~ ^($allowed_cmds)$ ]]; then
    $SSH_ORIGINAL_COMMAND
else
    echo "Comando non permitido: $user_cmd"
    exit 1
fi
```

→ Ya he explicado en el propio guión qué hace cada sección.

-Comandos que no se deben agregar para realizar una restricción óptima:

nano, vim, vi, ed, tee, cp, mv, rm → permiten escribir/modificar/eliminar
 scp, sftp, wget, curl, rsync → permitir carga/descarga de archivos
 tar, zip, unzip → puede escribir archivos al extraer
 bash, sh, python, perl, awk, sed → puede abrir shell o ejecutar scripts
 sudo → evidentemente...

→ Le damos permisos al script:

```
aso@base:~$ sudo chmod +x /usr/local/bin/ssh_readonly_shell.sh
```

→ Editamos o ficheiro de configuracion de ssh:

```
aso@base:~$ sudo nano /etc/ssh/sshd_config
```


→ Accedemos e añadimos:

```
Match Group ssh_readonly
    ForceCommand /usr/local/bin/ssh_readonly_shell.sh
    PermitTTY yes
    AllowTcpForwarding no
    X11Forwarding no
```

→ Esto se aplica solo a los usuarios del grupo ssh_readonly y forzará el uso del shell restringido.

→ Reiniciamos o servicio:

```
aso@base:~$ sudo systemctl restart ssh
```

PARTE 2 (AVANZADA)

Además, para hacerlo más avanzado y restrictivo podemos utilizar SELinux RBAC (Role-Based Access Control) para asignar un rol con permisos reducidos a determinados usuarios SSH.

→ Instalamos los paquetes de políticas MLS:

```
aso@base:~$ sudo apt update
sudo apt install selinux-policy-mls selinux-policy-default
Rcb:1 http://security.debian.org/debian-security bookworm-security InRelease [48,0 kB]
Teño:2 http://deb.debian.org/debian bookworm InRelease
Rcb:3 http://deb.debian.org/debian bookworm-updates InRelease [55,4 kB]
Rcb:4 http://security.debian.org/debian-security bookworm-security/main Sources [152 kB]
Rcb:5 http://security.debian.org/debian-security bookworm-security/main amd64 Packages [7
```

→ Ahora vamos a realizar un restorecon, para que cada archivo y directorio dentro de ellos, compare su contexto de seguridad SELinux actual con el contexto que debería tener según las políticas SELinux instaladas en el sistema. Si hay una diferencia, restorecon restaura

```
aso@base:~$ sudo restorecon -Rv /etc /home /var
Relabeled /etc/monit from unconfined_u:object_r:etc_t:s0 to unconfined_u:object_r:etc_t:s0
Relabeled /etc/monit/conf-available from unconfined_u:object_r:etc_t:s0 to unconfined_u:object_r:etc_t:s0
Relabeled /etc/monit/monitrc.d from unconfined_u:object_r:etc_t:s0 to unconfined_u:object_r:etc_t:s0
Relabeled /etc/monit/monitrc.d/fail2ban from unconfined_u:object_r:etc_t:s0 to unconfined_u:object_r:etc_t:s0
Relabeled /etc/selinux/mls/contexts from unconfined_u:object_r:selinux_t_r:default_context_t:s0 to unconfined_u:object_r:selinux_t_r:default_context_t:s0
Relabeled /etc/selinux/mls/contexts/virtual_domain_context from unconfined_u:object_r:selinux_t_r:default_context_t:s0 to unconfined_u:object_r:selinux_t_r:default_context_t:s0
```

→ Primero, verificamos los roles disponibles con semanage login -l:

```
aso@base:~$ sudo semanage login -l
```

Login Name	SELinux User	MLS/MCS Range	Servizo
__default__	user_u	s0-s0	*
root	root	s0-s15:c0.c1023	*
sddm	xdm	s0-s0	*

→ Ahora utilizaremos o usuario martin1

-SELinux incluye roles como:

usuario_u: usuario estándar (este es el que tienes por defecto)

personal_u: usuario con capacidad de escalar a root

invitado_u: función muy limitada (lectura básica, no se puede instalar, no se pueden cargar archivos)

xguest_u: similar, pero pensado para escritorio

→ Usaremos guest_u, ideal para SSH restringido, revisaremos los usuarios de SELinux:

```
aso@base:~$ sudo semanage user -l
```

SELinux User	Labeling Prefix	MLS/MCS Level	MLS/MCS Range	SELinux Roles
guest_u	user	s0	s0	user_r
root	sysadm	s0	s0-s15:c0.c1023	auditadm_r secadm_r staff_r sysadm_r
system_r				
staff_u	staff	s0	s0-s15:c0.c1023	auditadm_r secadm_r staff_r sysadm_r
sysadm_u	sysadm	s0	s0-s15:c0.c1023	sysadm_r
system_u	user	s0	s0-s15:c0.c1023	system_r
unconfined_u	unconfined	s0	s0-s15:c0.c1023	system_r unconfined_r
user_u	user	s0	s0	user_r
xdm	user	s0	s0	xdm_r

→ Ahora procederemos a asignar el rol SELinux user_u al usuario martin1:

```
aso@base:~$ sudo semanage login -a -s guest_u martin1
```

→ Verificar a asignación do rol:

```
aso@base:~$ sudo semanage login -l
```

Login Name	SELinux User	MLS/MCS Range	Servizo
__default__	user_u	s0-s0	*
martin1	guest_u	s0	*
root	root	s0-s15:c0.c1023	*
sddm	xdm	s0-s0	*

El rol guest_u ya está asignado exitosamente en SELinux. Tendrá las restricciones de acceso definidas por la política de SELinux para el usuario guest_u, que suelen ser muy limitadas para fines de seguridad y aislamiento.

13. Utilice SELinux para hacer chroot a los usuarios SSH

Un chroot é un mecanismo que limita os recursos que pode acceder un usuario. Isto pode ser útil para usuarios que só necesitan acceso a recursos moi específicos e debe impedirse que accedan a outras partes do sistema.

→ Accedemos o arquivo de configuración de SSH, e añadimos a seguinte directiva:

```
# Example of overriding settings on a per-user basis
Match User martin1
    ChrootDirectory /home/martin1
    AllowTcpForwarding no
    ForceCommand /bin/bash
    X11Forwarding no
```

→ La directiva para el usuario martin1:

- Encarcelarlo en su directorio de inicio (/home/martin1) usando una "cárcel chroot", restringiendo su acceso al sistema de archivos.
- Evita que se realice una tunelización TCP a través de la conexión SSH.
- Te obliga a iniciar un shell Bash interactivo dentro de tu entorno "chroot" cuando te conectas.
- Evita el reenvío de aplicaciones gráficas X11 a través de la conexión SSH.

→ Ahora temos que crear os directorios necesarios para que funcione chroot:

```
aso@base:~$ sudo chown root:root /home/martin1
sudo chmod 755 /home/martin1
```

→ El comando /bin/bash debe existir dentro del chroot

```
aso@base:~$ sudo mkdir -p /home/martin1/bin
sudo cp /bin/bash /home/martin1/bin/
```

→ As bibliotecas necesarias para bash tamén deben estar dentro do chroot

```
aso@base:~$ ldd /bin/bash
linux-vdso.so.1 (0x00007ffce93a3000)
libtinfo.so.6 => /lib/x86_64-linux-gnu/libtinfo.so.6 (0x00007fad2b703000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007fad2b522000)
/lib64/ld-linux-x86-64.so.2 (0x00007fad2b88b000)
```

→ Para eso imos generar un script automatizado:

```
GNU nano 7.2                                instalar_bash.sh
#!/bin/bash

CHROOT_DIR="/home/martin1"

echo "[+] Copiando /bin/bash a $CHROOT_DIR/bin"
mkdir -p "$CHROOT_DIR/bin"
cp /bin/bash "$CHROOT_DIR/bin"

echo "[+] Analizando dependencias de bash..."
libs=$(ldd /bin/bash | awk '{print $3}' | grep "^/")

echo "[+] Copiando librerías necesarias..."
for lib in $libs; do
    dest_dir="$CHROOT_DIR$(dirname $lib)"
    mkdir -p "$dest_dir"
    cp "$lib" "$dest_dir"
    echo "    Copiado: $lib → $dest_dir"
done

echo "[+] Copia completada. Bash e dependencias están no entorno chroot."

# Verificación rápida
echo -e "\n[+] Verifica que existe:"
ls -l "$CHROOT_DIR/bin/bash"
```

→ Este script automatiza los pasos necesarios para crear un entorno chroot mínimo para el usuario martin1 que incluye el shell Bash y sus bibliotecas esenciales. Este es un paso crucial para que la directiva ChrootDirectory y ForceCommand /bin/bash en el archivo de configuración SSH funcionen correctamente, permitiendo al usuario martin1 tener un shell interactivo pero restringido a su directorio de inicio.

→ Damosle permisos:

```
aso@base:~$ sudo chmod +x instalar_bash.sh
```

→ Ejecutámolo:

```
aso@base:~$ sudo ./instalar_bash.sh
[+] Copiando /bin/bash a /home/martin1/bin
[+] Analizando dependencias de bash...
[+] Copiando librerías necesarias...
    Copiado: /lib/x86_64-linux-gnu/libtinfo.so.6 → /home/martin1/lib/x86_64-linux-gnu
    Copiado: /lib/x86_64-linux-gnu/libc.so.6 → /home/martin1/lib/x86_64-linux-gnu
[+] Copia completada. Bash e dependencias están no entorno chroot.

[+] Verifica que existe:
-IWX-----. 1 root root 1265648 Abr 14 17:39 /home/martin1/bin/bash
```

→ Tras esto reiniciamos SSH

```
iso@base:~$ sudo systemctl restart ssh
```

14. Auditoría de comandos con auditd (SELinux-aware)

Instalar y configurar el sistema de auditoría de Linux auditd, que registra los comandos ejecutados por los usuarios, especialmente aquellos que afectan al sistema, y se integra perfectamente con SELinux. Esto permite monitorear y rastrear eventos críticos incluso cuando SELinux no bloquea, pero sí genera advertencias.

→ Instalamos:

```
aso@base:~$ sudo apt install auditd audispd-plugins
Lendo as listas de paquetes... Feito
Construindo a árvore de dependencias... Feito
Lendo a informação do estado... Feito
auditd is already the newest version (1:3.0.9-1).
```

→ Comprobamos o UID de martin1:

```
aso@base:~$ id martin1
uid=1002(martin1) gid=1003(martin1) grupos=1003(martin1)
```

→ Habilitar la auditoría de la ejecución de comandos por parte de los usuarios (monitor martin1)

```
aso@base:~$ sudo auditctl -a always,exit -F arch=b64 -F euid=1002 -S execve -k comandos_martin1
```

-Este comando configura o sistema de auditoría para:

- Grabe siempre la salida de cualquier llamada al sistema ejecutivo.
- Sólo se aplica a procesos que se ejecutan en arquitectura de 64 bits (arch=b64).
- Están siendo ejecutados por un usuario con un ID de usuario efectivo de 1002.
- Todos os rexistros generados por esta regla sexan etiquetados ca clave

comandos_martin1

→ Aquí es donde podemos comprobar los registros una vez que el usuario los ha ingresado:

```
aso@base:~$ sudo ausearch -k comandos_martin1
----
time->Tue Apr 15 11:14:03 2025
type=PROCTITLE msg=audit(1744708443.518:9419): proctitle=617564697463746C002D6100616C776179732C657869740
02D46006172636800623634002D4600657569640031303032002D5300657865637665002D6B00636F6D616E646F735F6D6172746
96E31
type=SOCKADDR msg=audit(1744708443.518:9419): saddr=1000000000000000000000000000000000
type=SYSCALL msg=audit(1744708443.518:9419): arch=c000003e syscall=44 success=yes exit=1072 a0=4 a1=7ffd
fb74f9b0 a2=430 a3=0 items=0 ppid=13019 pid=13020 auid=1000 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgi
d=0 fsgid=0 tty=pts1 ses=3 comm="auditctl" exe="/usr/sbin/auditctl" subj=unconfined_u:unconfined_r:audit
ctl_t:s0-s0:c0.c1023 key=(null)
type=CONFIG_CHANGE msg=audit(1744708443.518:9419): auid=1000 ses=3 subj=unconfined_u:unconfined_r:auditc
tl_t:s0-s0:c0.c1023 op=add_rule key="comandos_martin1" list=4 res=1
```

→ Lo haremos persistente después de reiniciar, para eso tenemos que acceder al archivo de configuración de reglas persistentes auditadas:

```
aso@base:~$ sudo nano /etc/audit/rules.d/comandos_martin1.rules
```

→ Añadimos a regla:

```
GNU nano 7.2 /etc/audit/rules.d/comandos_martin1.rules
-a always,exit -F arch=b64 -F euid=1002 -S execve -k comandos_martin1
```

→ Verificamos que la regla está activa:

```
aso@base:~$ sudo auditctl -l
-a always,exit -F arch=b64 -S execve -F euid=1002 -F key=comandos_martin1
```

La regla se carga en el sistema y registrará los comandos ejecutados por el usuario martin1

Esto sería para poder revisar los comandos de martin1 introducidos como usuario no servidor debian. Para poder auditar al usuario martin1 en la conexión SSH desde un cliente sería así:

→ Accedemos al archivo de reglas de audit e añadimos:

```
GNU nano 7.2 /etc/audit/rules.d/audit.rules *
## First rule - delete all
-D

## Increase the buffers to survive stress events.
## Make this bigger for busy systems
-b 8192

## This determine how long to wait in burst of events
--backlog_wait_time 60000

## Set failure mode to syslog
-f 1

# Reglas para sshd (por defecto)
-w /var/log/secure -p wa -k ssh_login
-w /var/log/auth.log -p wa -k ssh_login
-a always,exit -F path=/usr/sbin/sshd -F perm=x -k sshd_exec
-a always,exit -F arch=b64 -S connect -F auid>=1000 -F auid!=4294967295 -F path=/var/run/sshd.sock -k s
-a always,exit -F arch=b64 -S accept -F auid>=1000 -F auid!=4294967295 -F key=ssh_access
-a always,exit -F arch=b64 -S fork -F auid>=1000 -F auid!=4294967295 -F key=ssh_access

# Regla ESPECÍFICA para martin1
-a always,exit -F arch=b64 -F uid=1002 -S execve -k comandos_martin1

# Regla xeral para execve doutros usuarios (se é necesario)
-a always,exit -F arch=b64 -S execve -F auid>=1000 -F auid!=4294967295 -k execve_otros
```

Regla ESPECÍFICA para martin1

-a siempre,salir -F arch=b64 -F uid=1002 -S execve -k comandos_martin1

Ejecución del programa de auditoría (64 bits) realizada por el usuario martin1 (UID 1002), etiquetando los eventos como martin1_commands.

Regla general para la ejecución de otros usuarios (si es necesario)

-a siempre,salir -F arch=b64 -S execve -F auid>=1000 -F auid!=4294967295 -k execve_others

Ejecución del programa de auditoría (64 bits) por parte de usuarios reales conectados (según su ID de auditoría), etiquetando eventos como execve_others

15. Bloquear conexiones TCP salientes con nftables (Firewall de salida)

Con nftables puedes bloquear las conexiones TCP salientes, por ejemplo, permitiendo solo las necesarias (como SSH) y bloqueando el resto. Limita la capacidad de un proceso o usuario para establecer conexiones externas no autorizadas. Esencial en entornos de alto control como servidores compartidos o usuarios invitados.

→ Entonces, creamos la regla nftables que hace lo anterior:

```
aso@base:~$ sudo nft add rule inet filter output meta skuid martin tcp dport != 2222 drop
```

El gobernante hace lo siguiente: bloquea (descarta) todo el tráfico TCP saliente generado por el usuario martin1, excepto el tráfico con puerto de destino 2222.

→ Comprobamos que realmente foi añadida:

```
aso@base:~$ sudo nft list ruleset

chain output {
    type filter hook output priority filter; policy accept;
    meta skuid 1002 tcp dport != 2222 drop
}
```

→ Uha vez comprobado, reiniciamos Nftables

```
aso@base:~$ sudo systemctl restart nftables
```

También se han considerado más opciones de fortificación de SSH con SELinux, como habilitar el booleano deny_tcp_connect para evitar conexiones TCP salientes, pero esta opción no está disponible en la política actual de SELinux. En su lugar, los permisos de red se han restringido utilizando el usuario guest_u de SELinux, que no permite abrir sockets, y se puede complementar con una regla de iptables para evitar el tráfico saliente de usuarios restringidos.

7. PRUEBAS DE FORTIFICACIÓN DE SSH

En primer lugar, comprobamos que la máquina se conecta con el cliente Kali:

→De Debian a Kali:

```
aso@base:~$ ping 192.168.56.100
PING 192.168.56.100 (192.168.56.100) 56(84) bytes of data.
64 bytes from 192.168.56.100: icmp_seq=1 ttl=64 time=1.60 ms
64 bytes from 192.168.56.100: icmp_seq=2 ttl=64 time=0.727 ms
64 bytes from 192.168.56.100: icmp_seq=3 ttl=64 time=0.953 ms
^C
--- 192.168.56.100 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2005ms
rtt min/avg/max/mdev = 0.727/1.093/1.601/0.370 ms
```

→ Dos veces en debian:


```
(kali㉿kali)-[~/Desktop]
$ ping 192.168.56.104
PING 192.168.56.104 (192.168.56.104) 56(84) bytes of data.
64 bytes from 192.168.56.104: icmp_seq=1 ttl=64 time=0.882 ms
64 bytes from 192.168.56.104: icmp_seq=2 ttl=64 time=4.65 ms
64 bytes from 192.168.56.104: icmp_seq=3 ttl=64 time=3.23 ms
64 bytes from 192.168.56.104: icmp_seq=4 ttl=64 time=5.77 ms
64 bytes from 192.168.56.104: icmp_seq=5 ttl=64 time=3.74 ms
64 bytes from 192.168.56.104: icmp_seq=6 ttl=64 time=3.22 ms
^C
--- 192.168.56.104 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5007ms
rtt min/avg/max/mdev = 0.882/3.580/5.765/1.497 ms
```

→Desde Kali también podemos realizar un escaneo nmap para comprobar qué detecta la máquina:

```
(kali㉿kali)-[~/Desktop]
$ ping 192.168.56.104
PING 192.168.56.104 (192.168.56.104) 56(84) bytes of data.
64 bytes from 192.168.56.104: icmp_seq=1 ttl=64 time=0.882 ms
64 bytes from 192.168.56.104: icmp_seq=2 ttl=64 time=4.65 ms
64 bytes from 192.168.56.104: icmp_seq=3 ttl=64 time=3.23 ms
64 bytes from 192.168.56.104: icmp_seq=4 ttl=64 time=5.77 ms
64 bytes from 192.168.56.104: icmp_seq=5 ttl=64 time=3.74 ms
64 bytes from 192.168.56.104: icmp_seq=6 ttl=64 time=3.22 ms
^C
--- 192.168.56.104 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5007ms
rtt min/avg/max/mdev = 0.882/3.580/5.765/1.497 ms
```

Como podemos ver, detecta la máquina .104

→ Ahora veamos qué puertos están abiertos para Debian:

```
(kali@kali)~[~/Desktop]
$ sudo nmap -n -Pn -sV -A -O -p- 192.168.56.104
Starting Nmap 7.95 ( https://nmap.org ) at 2025-04-24 16:20 CEST
Nmap scan report for 192.168.56.104
Host is up (0.00084s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE      SERVICE VERSION
22/tcp    filtered  ssh
2222/tcp  open      ssh      OpenSSH 9.2p1 Debian 2+deb12u5 (protocol 2.0)
| ssh-hostkey:
|_ 256 28:80:5f:25:e8:aa:42:5c:2d:fc:f6:88:5d:4f:ef:a8 (ECDSA)
|_ 256 b6:78:c8:5f:67:f8:e7:ab:22:87:5a:bc:48:64:ba:71 (ED25519)
MAC Address: 08:00:27:FF:99:87 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Device type: general purpose
Running: Linux 4.X|5.X
OS CPE: cpe:/o:linux:linux_kernel:4 cpe:/o:linux:linux_kernel:5
OS details: Linux 4.15 - 5.19, OpenWrt 21.02 (Linux 5.4)
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

TRACEROUTE
HOP RTT      ADDRESS
1   0.83 ms  192.168.56.104

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.76 seconds
```

Como podemos observar no escaneo:

-Puerto 22 ssh filtrado, significa que el puerto 22 no responde al polo de escaneo que está bloqueado o cancelado. Esto significa que confirmamos que la fortificación está funcionando.

- Abra el puerto ssh 2222, que fue el puerto que agregamos en las reglas de SELinux

AHORA COMENCEMOS A PROBAR LAS REGLAS:

1. Desactivamos o acceso root por SSH.

[vídeo proba acceso root](#)

Para verificar la correcta implementación de la directiva de seguridad que deshabilita el acceso root mediante SSH, se intentó conectarse al servidor utilizando el usuario root a través del puerto no estándar configurado (2222). Aunque el cliente SSH solicitó una contraseña, el análisis de los registros del servidor SSH reveló entradas de "error de autenticación" para el usuario root, lo que confirma que el acceso directo (incluso ingresando correctamente la contraseña root) está prohibido.

En el propio vídeo podemos ver que solo podemos acceder al root en el servidor SSH, por lo que podemos comprobar que la contraseña es correcta y funciona.

2. Reforzamos el uso de claves SSH en lugar de contraseñas.

[video rechazo por claves SSH](#)

Para verificar que el sistema exige el uso de claves SSH y no permite la autenticación basada en contraseña, se realizó una prueba desde la máquina cliente (Kali Linux) sin proporcionar una clave privada válida para el usuario aso en el servidor (conectándose al puerto no estándar 2222).

Ahora imos intentar acceder como accedería un usuario permitido:

→ Generamos a clave

```
(kali@kali)-[~/Desktop]
$ ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/home/kali/.ssh/id_rsa):
Enter passphrase for "/home/kali/.ssh/id_rsa" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/kali/.ssh/id_rsa
Your public key has been saved in /home/kali/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:CRujU/V3KAyrwGnUG/yJMkkKVeM5iBesgFnDw0jQ/j8 kali@kali
The key's randomart image is:
+---[RSA 4096]-----+
|O=.+o  o          |
|Bo+B.++. = .      |
|+=+oo0.==..+ o .  |
|.oo.++=o*o. o .   |
|  .ooo S          |
|  ..              |
|  .               |
|  E              |
|  .               |
+---[SHA256]-----+
```

→ Comprobamos:

```
(kali@kali)-[~/ssh]
$ ls
id_rsa  id_rsa.pub  known_hosts  known_hosts.old
```

→ Copiamos la clave

```
(kali@kali)-[~/ssh]
$ cat id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCAQCvFL6/N0Whjx0GrthBs/V2KnnkICiFDvwcJdhgU8Mj1wS09rK2ApX/0NZBpneTc2k+75+2miP++qv3
z8ixV8voERn/Ji+Hakr1kle0cp+tSlfCOp0S1zHfgiKwLTgxSNw5LWxs182sVKfvJBtuEjcYtmMyMLEMeZKIHERrt/GuUxmnd6ZIKa/DDnu5h+lpzLzk
z4s65N/M21bd/GN+u1d00fivqrCwnJPqp2519LjbxYHsUKagPAYVgvN0tCXy3/BrDcnJamr6u2uNmBd/x8TAAem5r1zBA9Pg4ngmKZjzZCKUKA2VFK
RfF+pkkWhykRvGISFe+qkErLWWJoJDM/mQIR48ii17q7ABg2Ns3JAww712YX50W/bm85Iqz87qxtJ0FH5FkuCL3eJDvivrPGurWdv+xEIDLbVl+HtyCmw
Y/bhtl9jKgL9dmh7EotQSjfkLwto0dv6MZ0YLUunuU9A9xyQaEM8++4dxF4SNnTB2nxjtRIh0GbD340W42f9Efar/UwogxmIWuYbhE/p5oxhs1l/XnEX
45slb2/iUI689n1f8GD2XbMpyLK+wz4XaGYuhbRtrRDLmZ3PPLm7fILa2azlpaZkKWiw0Dp/c08mDfSGK+gY4qaG2MNkzgTgiPuhv19jjidjnzUoIgtte
Pwiwji/EoKJSnQYh6h0BnF8e6w== kali@kali
```

→ Vamos al servidor ssh y pegamos la clave pública en:

```
aso@base:~/ssh$ sudo nano authorized_keys
```

→ Una vez atascado, devolvemos el kali y probamos:

[video clave SSH Si](#)

3. Cambiamos el puerto a un número alto y lo registramos en SELinux.

[puerto de video 22 en 2222 si](#)

Como podemos ver, el intento de conexión a través del puerto 22, que es el puerto SSH por defecto, no se puede realizar. Sin embargo, la conexión sólo podrá realizarse a través del nuevo puerto.

4. Limitamos los usuarios que pueden iniciar sesión.

[límite de vídeo para usuarios SSH](#)

→ Configuración anterior no archivo de configuración de ssh:

```
AllowUsers aso martin martin1 martin2
```

-Al intentar conectarse a un usuario incluido en la lista AllowUsers (martin1), la conexión fue denegada con un Permiso denegado (clave pública). El análisis de los registros del servidor no mostró un error de usuario no válido para este intento, lo que sugiere que se permitió la conexión a nivel de usuario, pero la autenticación falló debido a la falta de una clave pública válida del cliente.

-Al intentar conectarse a un usuario no incluido en la lista AllowUsers (nugget), la conexión también resultó en un Permiso denegado (clave pública) en el cliente. Sin embargo, el análisis de los registros del servidor reveló el mensaje "Usuario inválido pepito". Esto confirma que la directiva AllowUsers está funcionando para rechazar a usuarios no autorizados en una etapa temprana de la conexión, antes de que se intente la autenticación.

5. Configuramos un firewall que sólo permite conexiones SSH seguras.

```
(kali@kali)~[~/Desktop]
$ sudo nmap -n -Pn -sV -A -O -p- 192.168.56.104
Starting Nmap 7.95 ( https://nmap.org ) at 2025-04-24 16:20 CEST
Nmap scan report for 192.168.56.104
Host is up (0.00084s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE      SERVICE VERSION
22/tcp    filtered  ssh
2222/tcp  open      ssh      OpenSSH 9.2p1 Debian 2+deb12u5 (protocol 2.0)
```

En la captura que vimos anteriormente:

-Puerto 22 filtrado: Esto indica que el firewall está bloqueando activamente las conexiones entrantes al puerto SSH estándar (puerto 22).

-Puerto 2222 abierto: Este es el puerto SSH no estándar que configuré, y el estado abierto indica que nmap pudo establecer exitosamente una conexión TCP en ese puerto, lo que confirma que el servicio SSH está escuchando correctamente en el puerto 2222 y que el firewall permite conexiones a ese puerto.

6. Engañamos a Fail2Ban para bloquear intentos de acceso sospechosos.

[Vídeo Fail2Ban](#)

Para verificar la funcionalidad de Fail2Ban en la protección del servicio SSH, se realizó una prueba simulando intentos de acceso sospechosos desde la máquina cliente (Kali Linux). La cárcel SSH se configuró con un límite de intentos fallidos (maxretry) dentro del período de tiempo predeterminado (findtime), después del cual la dirección IP del atacante se bloquearía durante 600 segundos (bantime = 600).

7. Habilitamos SELinux MLS para un control más estricto.

```
aso@base:~$ sestatus
SELinux status:                enabled
SELinuxfs mount:              /sys/fs/selinux
SELinux root directory:       /etc/selinux
Loaded policy name:            mls
Current mode:                  enforcing
Mode from config file:         enforcing
Policy MLS status:             enabled
Policy deny_unknown status:    denied
Memory protection checking:    actual (secure)
Max kernel policy version:    33
```

La imagen superior muestra el estado de SELinux en el servidor después de haber sido configurado y reiniciado. La salida del comando sestatus indica que SELinux está habilitado y que la política de seguridad mls (seguridad multinivel) está cargada. El modo actual es de cumplimiento, lo que significa que SELinux está aplicando activamente las políticas de seguridad y denegando acciones que no están explícitamente permitidas. El modo del archivo de configuración también muestra la aplicación, lo que confirma que esta configuración persistirá en futuros reinicios.

8. Monitoreamos eventos en SSH con SELinux y creamos políticas personalizadas.

[vídeo Monitorización con SELinux](#)

Este registro de auditoría de SELinux demuestra claramente que el intento de iniciar sesión como root a través de SSH fue registrado nuevamente por el sistema de auditoría. El campo res=failed confirma que la autenticación falló, como se esperaba debido a la configuración de PermitRootLogin en sshd_config

9. Implementamos 2FA para agregar una capa adicional de seguridad.

[Vídeo 2FA](#)

Para verificar la correcta implementación de la autenticación de dos factores (2FA) mediante Google Authenticator, se intentó iniciar sesión mediante SSH en el servidor (puerto 2222) con el usuario aso. Para esta prueba específica, se deshabilitó temporalmente la autenticación de clave SSH para permitir el acceso con contraseña (habilitado temporalmente) seguido de la solicitud del código de verificación 2FA.

Como se puede ver en el vídeo, el sistema solicitó un 'Código de verificación'. Este comportamiento confirma que el segundo factor de autenticación está activo y funcionando.

10. Implemente Port Knocking para ocultar el puerto SSH.

[Vídeo Puerto golpeando](#)

Implementé la técnica de port knocking con el objetivo de agregar una capa adicional de seguridad al ocultar el puerto de acceso SSH. De esta manera, el puerto permanece cerrado e invisible a los escaneos de red hasta que se envía una secuencia predefinida a otros puertos. Para verificar el correcto funcionamiento de esta implementación, envié desde la máquina cliente Kali la secuencia de puertos configurada (7000, 8000, 9000) al servidor Debian. Luego de enviar la secuencia correcta, verifiqué usando la herramienta netcat (nc -zv) que el puerto 2222 estaba abierto temporalmente, permitiendo así una conexión SSH

11. Establecer umask para restringir permisos por defecto.

[Vídeo UMASK](#)

Con el objetivo de evitar la creación de archivos con permisos inseguros por parte de usuarios que accedan vía SSH.

Para verificar la correcta aplicación de esta configuración accedí vía SSH. La salida del comando `umask` confirmó la activación de la máscara con el valor 077. Después, creé un archivo de prueba. Al inspeccionar los permisos del archivo creado con el comando `ls -l`, se observó que los permisos resultantes fueron `-rw----- (600)`.

De esta forma se asegura que todos los archivos y directorios creados por usuarios SSH tengan permisos restrictivos por defecto, fortaleciendo la seguridad del sistema.

12. Configure SSH Escape Timing para mitigar ataques de fuerza bruta.

La desconexión automática no requiere la intervención del usuario ni pruebas complejas más allá de verificar que, tras un periodo de inactividad determinado, la sesión finaliza correctamente, liberando recursos y minimizando posibles riesgos.

13. Restringir la ejecución de comandos a través de SSH

[video Restriccion Comandos](#)

La restricción de comando básica para el usuario `aso` que usa un script y `ForceCommand` está funcionando bien. Los comandos incluidos en la lista blanca se ejecutan y los comandos no permitidos se bloquean con un mensaje.

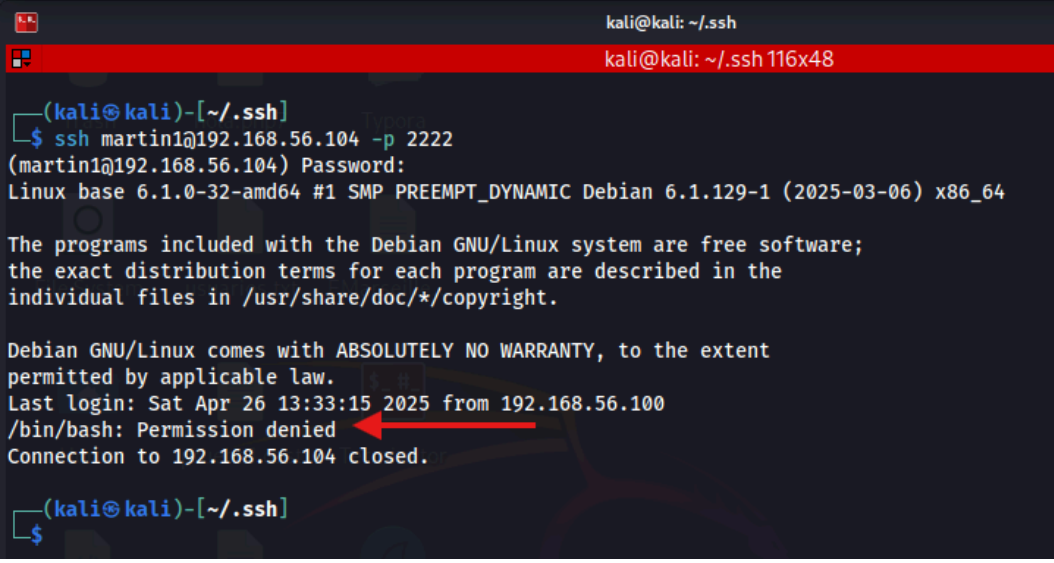
14. Utilice SELinux para hacer chroot a los usuarios SSH



```
aso@base: ~$ sudo chroot /home/martin1 /bin/bash
bash-5.2# pwd
/
bash-5.2# ls
bin  home  lib  lib64
bash-5.2# cd /bin
cd: cannot access '/bin'
bash-5.2# ls /lib
ls: cannot access '/lib'
bash-5.2#
```

Una vez dentro del entorno chroot configurado para `/home/martin1`, el usuario no puede navegar fuera de este directorio raíz virtual. Los intentos de utilizar `cd ..` para subir un nivel y acceder al directorio `/etc` (que está fuera del entorno chroot en el sistema real) fallan. Esto confirma que el chrooting del directorio raíz funciona como se esperaba. El usuario `martin1`, al operar dentro de este entorno, estará restringido al sistema de archivos contenido en

/home/martin1



```
kali@kali: ~/.ssh
kali@kali: ~/.ssh 116x48

(kali@kali)~[~/.ssh]
$ ssh martin1@192.168.56.104 -p 2222
(martin1@192.168.56.104) Password:
Linux base 6.1.0-32-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.129-1 (2025-03-06) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sat Apr 26 13:33:15 2025 from 192.168.56.100
/bin/bash: Permission denied
Connection to 192.168.56.104 closed.

(kali@kali)~[~/.ssh]
$
```

El problema que aún impide la conexión exitosa desde el cliente Kali sin la configuración chroot (y por lo tanto el problema real con el chroot a través de SSH) probablemente radica en una combinación de factores relacionados con los permisos y la configuración SSH específica para el entorno chroot.

Para que esto funcione, creé la estructura del directorio bin dentro del chroot y copié el shell bash y sus bibliotecas necesarias. Encontré y resolví problemas de permisos en el directorio /home/martin1/bin que impedían el acceso. Finalmente, prohibí el confinamiento del chroot localmente en el servidor usando el comando chroot, verificando que la navegación fuera de /home/martin1 (apareciendo como / dentro del chroot) estuviera restringida.

15. Auditoría de comandos con auditd (SELinux-aware)

[Video Auditoria de Comandos](#)

Esta auditoría nos permite tener un registro detallado de todas las acciones realizadas por el usuario martin1 durante su sesión SSH, lo cual es crucial para la seguridad y monitorización del sistema. En caso de incidentes o comportamientos sospechosos, podemos revisar estos registros para comprender las acciones tomadas por el usuario.

16. Bloquear conexiones TCP salientes con nftables (Firewall de salida)

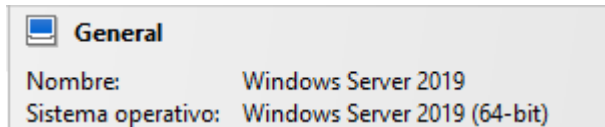
[video NfTables](#)

Si ve los resultados esperados en cada prueba, confirmará que el firewall de salida con nftables está funcionando correctamente para el usuario martin, bloqueando todas las conexiones TCP excepto el puerto 2222. El tráfico que no sea TCP (como ICMP) no se verá afectado por esta regla específica.

5. Ejecución de WINDOWS

1. CREACIÓN DAS MÁQUINAS VIRTUALES

Servidor Windows 2019

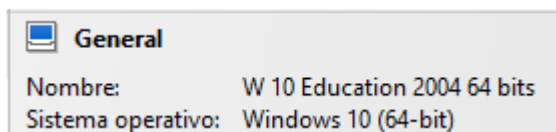


General

Nombre: Windows Server 2019

Sistema operativo: Windows Server 2019 (64-bit)

Cliente de Windows 10



General

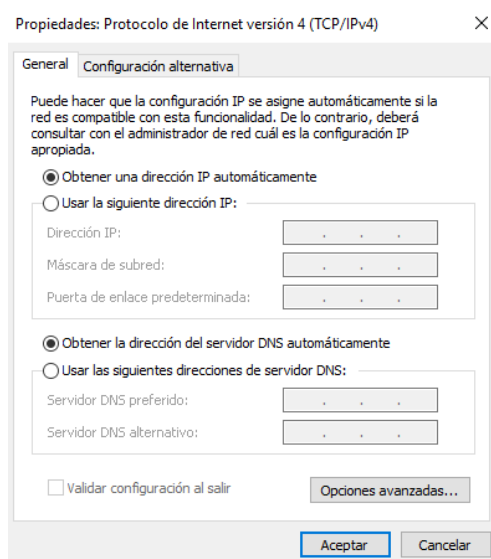
Nombre: W 10 Education 2004 64 bits

Sistema operativo: Windows 10 (64-bit)

2. CONFIGURACIÓN DE RED DE MÁQUINAS

CONFIGURACIÓN DE REDE DO WINDOWS SERVER

→adaptador nat:



Propiedades: Protocolo de Internet versión 4 (TCP/IPv4) X

General Configuración alternativa

Puede hacer que la configuración IP se asigne automáticamente si la red es compatible con esta funcionalidad. De lo contrario, deberá consultar con el administrador de red cuál es la configuración IP apropiada.

☒ Obtener una dirección IP automáticamente

☐ Usar la siguiente dirección IP:

Dirección IP: . . .

Máscara de subred: . . .

Puerta de enlace predeterminada: . . .

☒ Obtener la dirección del servidor DNS automáticamente

☐ Usar las siguientes direcciones de servidor DNS:

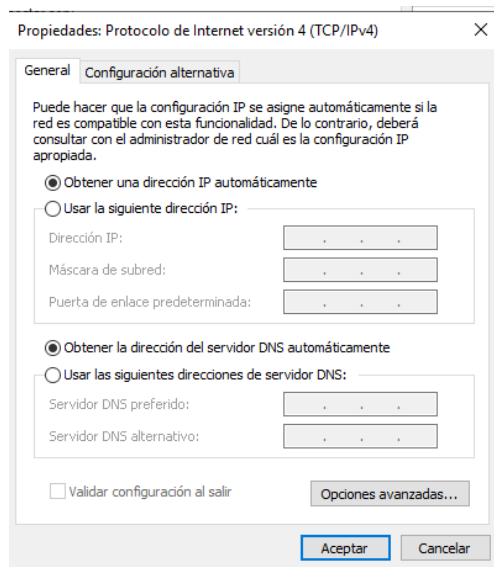
Servidor DNS preferido: . . .

Servidor DNS alternativo: . . .

☐ Validar configuración al salir Opciones avanzadas...

Aceptar Cancelar

→ adaptador solo anfitrión:



Ambos los dejamos en DHCP ya que el NAT me da la misma ip que adapta, y ya he puesto los intervalos en la configuración del virtual box para el adaptador host-only.

→ Comprobamos configuración ip:

```
C:\Users\Administrador>ipconfig

Configuración IP de Windows

Adaptador de Ethernet Ethernet:

    Sufrido DNS específico para la conexión. . . :
    Dirección IPv6 . . . . . : fd00::f0ae:f6df:7f0c:e2c5
    Vínculo: dirección IPv6 local. . . : fe80::f0ae:f6df:7f0c:e2c5%6
    Dirección IPv4. . . . . : 10.0.2.15
    Máscara de subred . . . . . : 255.255.255.0
    Puerta de enlace predeterminada . . . . : fe80::2%6
                                           10.0.2.2

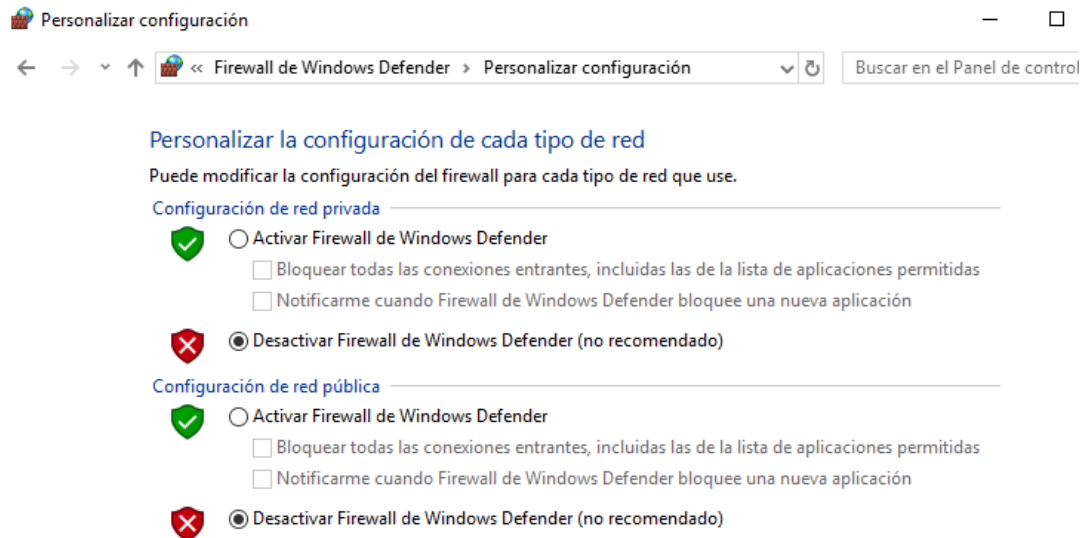
Adaptador de Ethernet Ethernet 2:

    Sufrido DNS específico para la conexión. . . :
    Vínculo: dirección IPv6 local. . . : fe80::5913:564d:205b:69d1%4
    Dirección IPv4. . . . . : 192.168.56.105
    Máscara de subred . . . . . : 255.255.255.0
    Puerta de enlace predeterminada . . . . :
```

Nat: 10.0.2.15

Solo Anfitrión: 192.168.56.105

→ En Windows debemos desactivar el Firewall de Windows Defender para poder recibir conexiones entrantes y salientes:



Si lo dejamos por defecto, estas conexiones serán rechazadas automáticamente.

→ hacer ping a google:

```
C:\Users\Administrador>ping 8.8.8.8

Haciendo ping a 8.8.8.8 con 32 bytes de datos:
Respuesta desde 8.8.8.8: bytes=32 tiempo=15ms TTL=255
Respuesta desde 8.8.8.8: bytes=32 tiempo=15ms TTL=255
Respuesta desde 8.8.8.8: bytes=32 tiempo=16ms TTL=255
Respuesta desde 8.8.8.8: bytes=32 tiempo=15ms TTL=255

Estadísticas de ping para 8.8.8.8:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
    (0% perdidos),
    Tiempos aproximados de ida y vuelta en milisegundos:
        Mínimo = 15ms, Máximo = 16ms, Media = 15ms
```

→ hacer ping a debian:

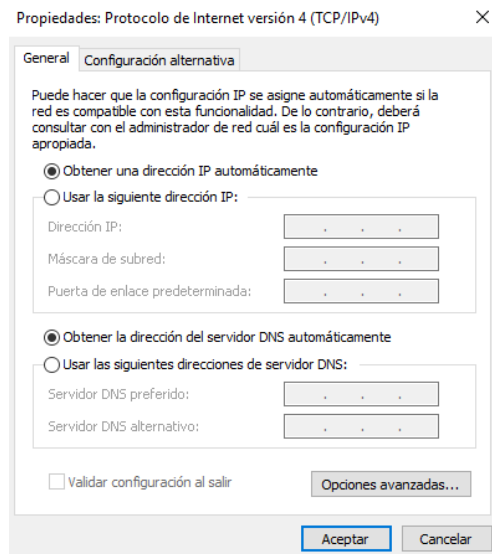
```
C:\Users\Administrador>ping 192.168.56.104

Haciendo ping a 192.168.56.104 con 32 bytes de datos:
Respuesta desde 192.168.56.104: bytes=32 tiempo<1m TTL=64
Respuesta desde 192.168.56.104: bytes=32 tiempo=1ms TTL=64
Respuesta desde 192.168.56.104: bytes=32 tiempo=1ms TTL=64
Respuesta desde 192.168.56.104: bytes=32 tiempo<1m TTL=64

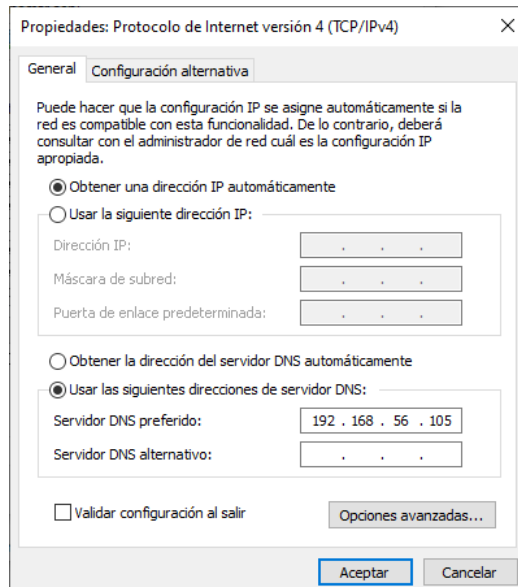
Estadísticas de ping para 192.168.56.104:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
    (0% perdidos),
    Tiempos aproximados de ida y vuelta en milisegundos:
        Mínimo = 0ms, Máximo = 1ms, Media = 0ms
```

CONFIGURACIÓN DE RED DEL CLIENTE DE WINDOWS 10

→adaptador nat:



→ adaptador solo anfitrión:



Al igual que el Windows Server, ambos los dejamos en DHCP ya que el NAT me da la misma ip que adapta, y ya he puesto los intervalos en la configuración del virtual box para el adaptador host-only. En este caso en DNS debemos poner la IP del Servidor Windows.

Al igual que en Win Server, también deshabilitamos el firewall para evitar bloqueos al realizar conexiones.

→ Configuración:

```
C:\Windows\system32>ipconfig

Configuración IP de Windows

Adaptador de Ethernet Ethernet:

    Sufijo DNS específico para la conexión. . . :
    Dirección IPv6 . . . . . : fd00::b190:3739:45be:e546
    Dirección IPv6 temporal. . . . . : fd00::e5b0:dbc1:6697:73cc
    Vínculo: dirección IPv6 local. . . : fe80::b190:3739:45be:e546%6
    Dirección IPv4. . . . . : 10.0.2.15
    Máscara de subred . . . . . : 255.255.255.0
    Puerta de enlace predeterminada . . . . : fe80::2%6
                                                10.0.2.2

Adaptador de Ethernet Ethernet 2:

    Sufijo DNS específico para la conexión. . . :
    Vínculo: dirección IPv6 local. . . : fe80::cc86:9ce0:aa4b:8ccf%13
    Dirección IPv4. . . . . : 192.168.56.106
    Máscara de subred . . . . . : 255.255.255.0
    Puerta de enlace predeterminada . . . . :

C:\Windows\system32>
```

→ Hacer ping a un servidor Windows:

```
C:\Windows\system32>ping 192.168.56.105

Haciendo ping a 192.168.56.105 con 32 bytes de datos:
Respuesta desde 192.168.56.105: bytes=32 tiempo<1m TTL=128
Respuesta desde 192.168.56.105: bytes=32 tiempo<1m TTL=128
Respuesta desde 192.168.56.105: bytes=32 tiempo<1m TTL=128
Respuesta desde 192.168.56.105: bytes=32 tiempo=1ms TTL=128

Estadísticas de ping para 192.168.56.105:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
    (0% perdidos),
    Tiempos aproximados de ida y vuelta en milisegundos:
        Mínimo = 0ms, Máximo = 1ms, Media = 0ms

C:\Windows\system32>_
```

→ Hacer ping a una red:

```
C:\Windows\system32>ping 192.168.56.105

Haciendo ping a 192.168.56.105 con 32 bytes de datos:
Respuesta desde 192.168.56.105: bytes=32 tiempo<1m TTL=128
Respuesta desde 192.168.56.105: bytes=32 tiempo<1m TTL=128
Respuesta desde 192.168.56.105: bytes=32 tiempo<1m TTL=128
Respuesta desde 192.168.56.105: bytes=32 tiempo=1ms TTL=128

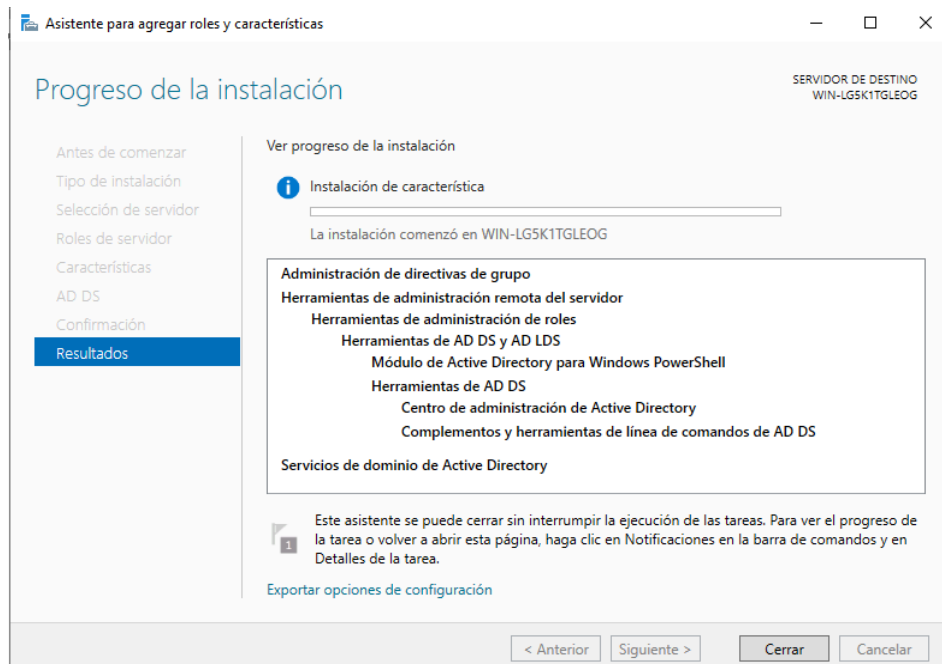
Estadísticas de ping para 192.168.56.105:
    Paquetes: enviados = 4, recibidos = 4, perdidos = 0
    (0% perdidos),
    Tiempos aproximados de ida y vuelta en milisegundos:
        Mínimo = 0ms, Máximo = 1ms, Media = 0ms

C:\Windows\system32>_
```

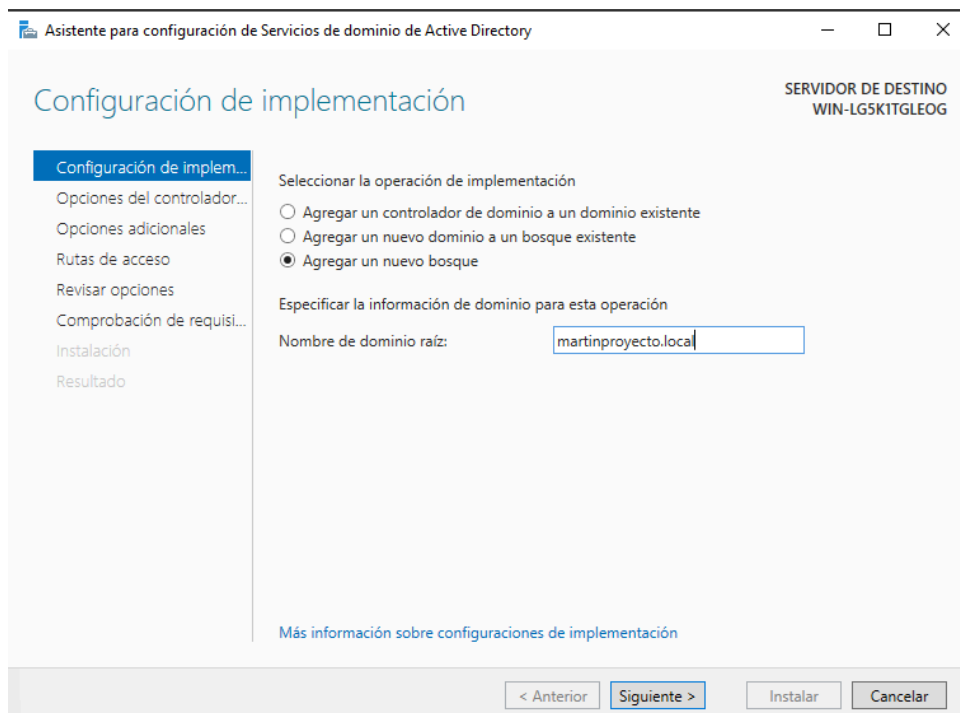
3. CONFIGURACIONES ANTERIORES

Vamos a crear un dominio en el Servidor Windows para poner el Cliente en él.

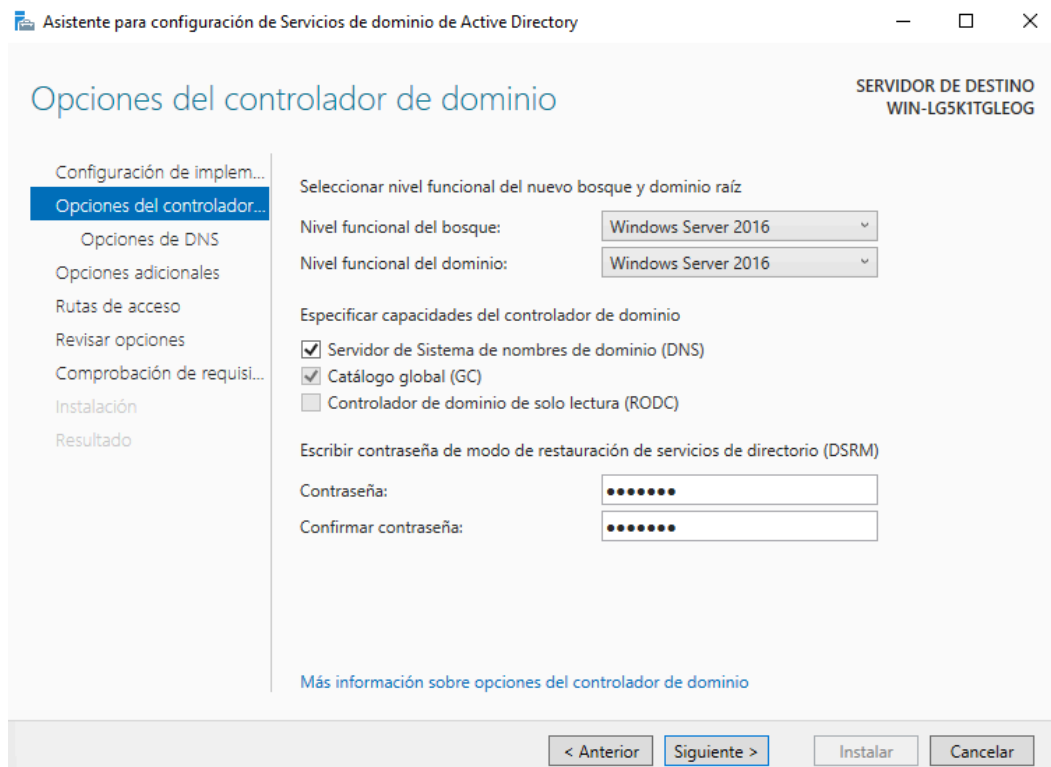
→ Para comenzar, temos que instalar os servicios de Active Directory



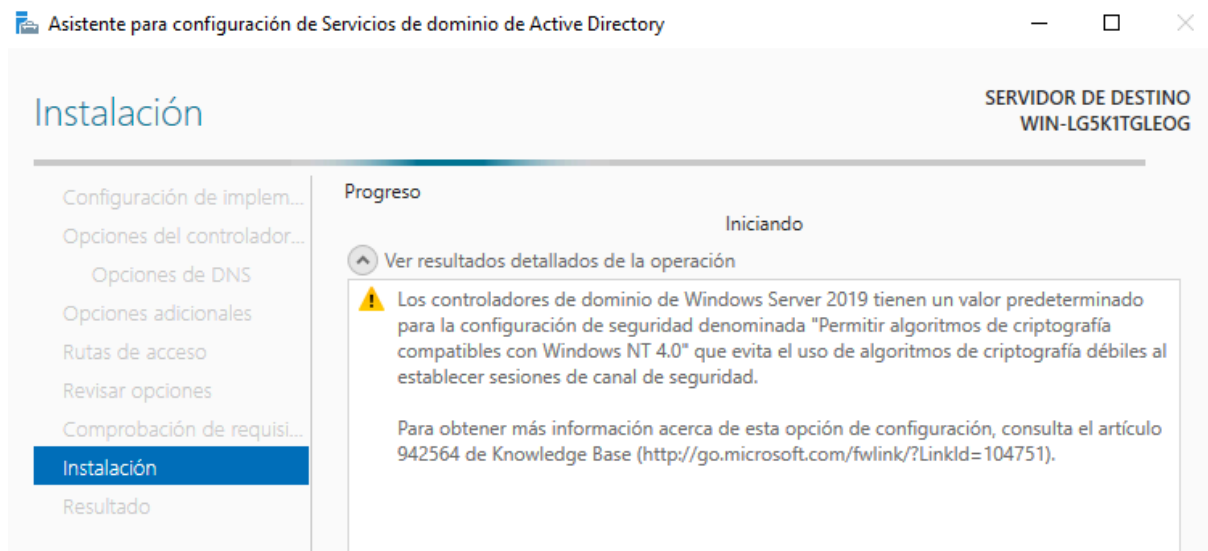
→ Ahora promovemos o Servidor a Servidor de Dominio e creamos o dominio



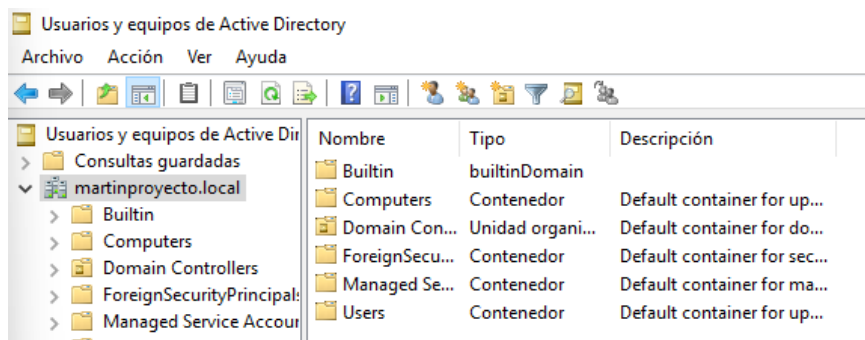
→ Seleccionamos o nivel funcional e marcamos a opción de DNS



→ Damosle a siguiente, e instalamos.



→ Comprobemos que se ha creado correctamente:



→ Ahora tenemos que meter o Cliente Windows no dominio que acabamos de crear:

Cambios en el dominio o el nombre del equipo X

Puede cambiar el nombre y la pertenencia de este equipo. Los cambios podrían afectar al acceso a los recursos de red.

Nombre de equipo:
pruebamartin

Nombre completo de equipo:
pruebamartin

Más...

Miembro del

☒ Dominio:
martinproyecto.local

☐ Grupo de trabajo:
WORKGROUP

Aceptar Cancelar

→ Verificamos:

Cambios en el dominio o el nombre del equipo X

 Se unió correctamente al dominio martinproyecto.local.

Aceptar

→ Ahora accedemos co Usuario ó cal lle vou aplicar las reglas de AppLocker

 Cambiar la configuración de la cuenta

 Bloquear

 Cerrar sesión

 Cambiar de usuario

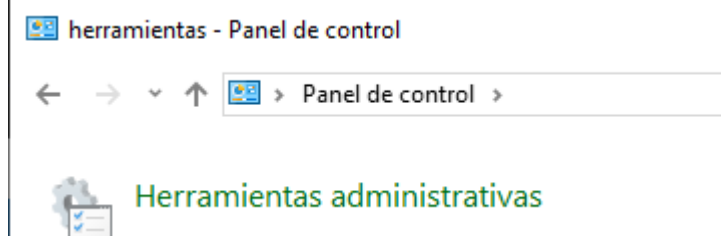
 pruebamartin

→ Comprobamos tamen no controlador de dominio que sí aparece:

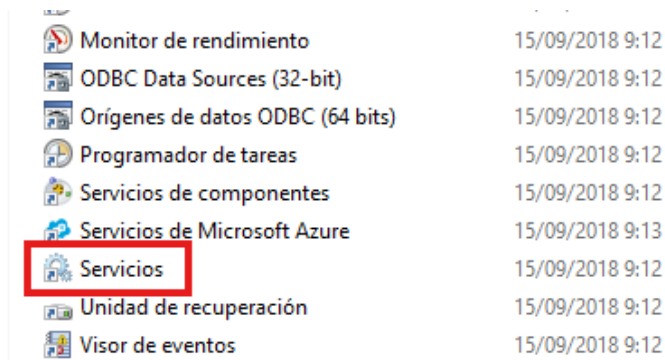
Usuarios y equipos de Active Dir	Nombre	Tipo
> Consultas guardadas		
▼ martinproyecto.local	 PRUEBAMARTIN	Equipo
Built-in		
Computers		
Domain Controllers		
ForeignSecurityPrincipal:		

Para comenzar, AppLocker requiere que el servicio "Identidad de la aplicación" esté habilitado.

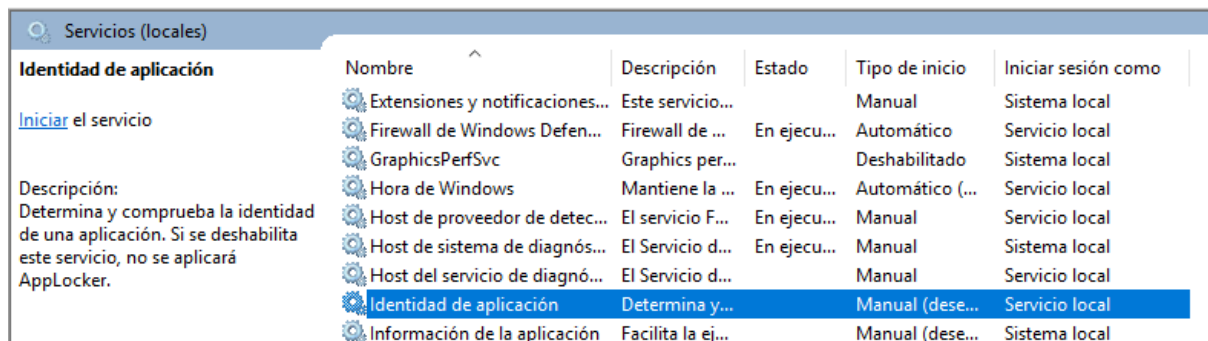
→ Habilitaremos el servicio de "Identidad de la aplicación". Tenemos que acceder al panel de control y acceder a las herramientas administrativas:



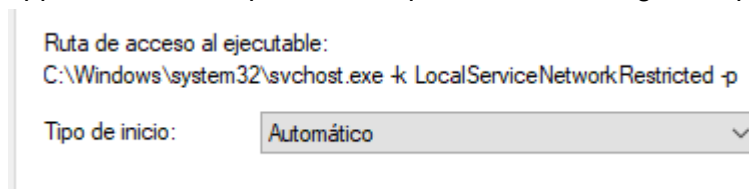
→ Una vez dentro, accedemos a los servicios:



→ Entramos a servicios, e buscamos identidad de aplicación:



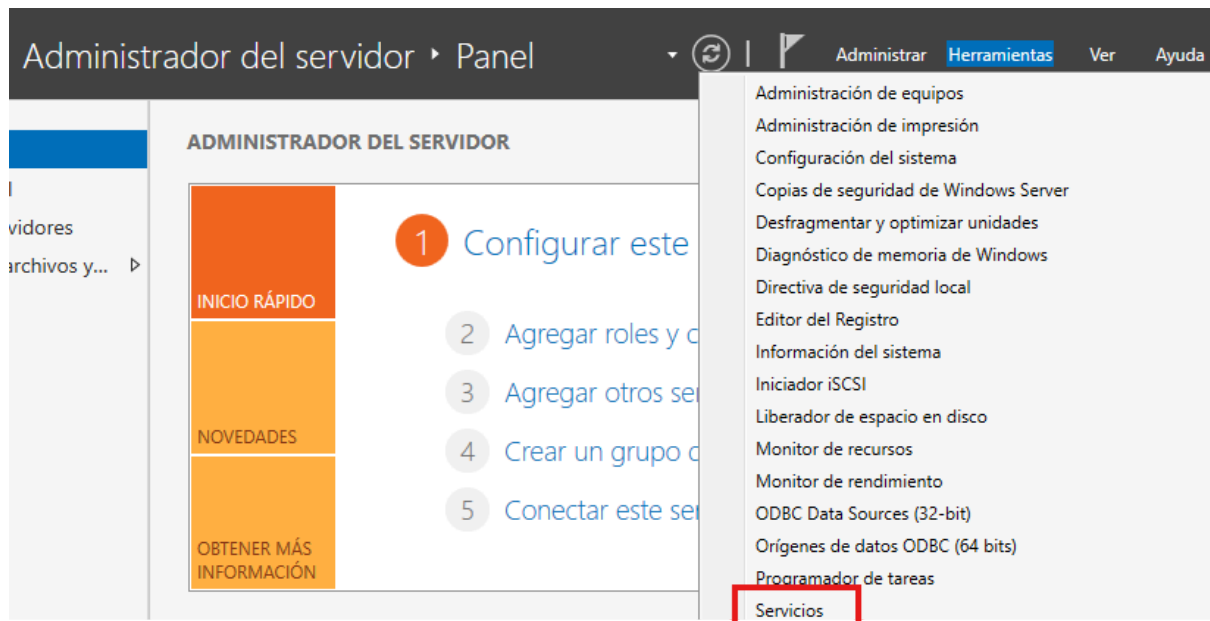
→ Como podemos ver, ya nos comentó en la descripción que sí está deshabilitado, AppLocker no se aplicará, así que vamos a configurarlo para que se inicie automáticamente:



→ E ahora imos inicial:



Tamén podríamos acceder directamente desde el Administrador del Servidor, en herramientas:

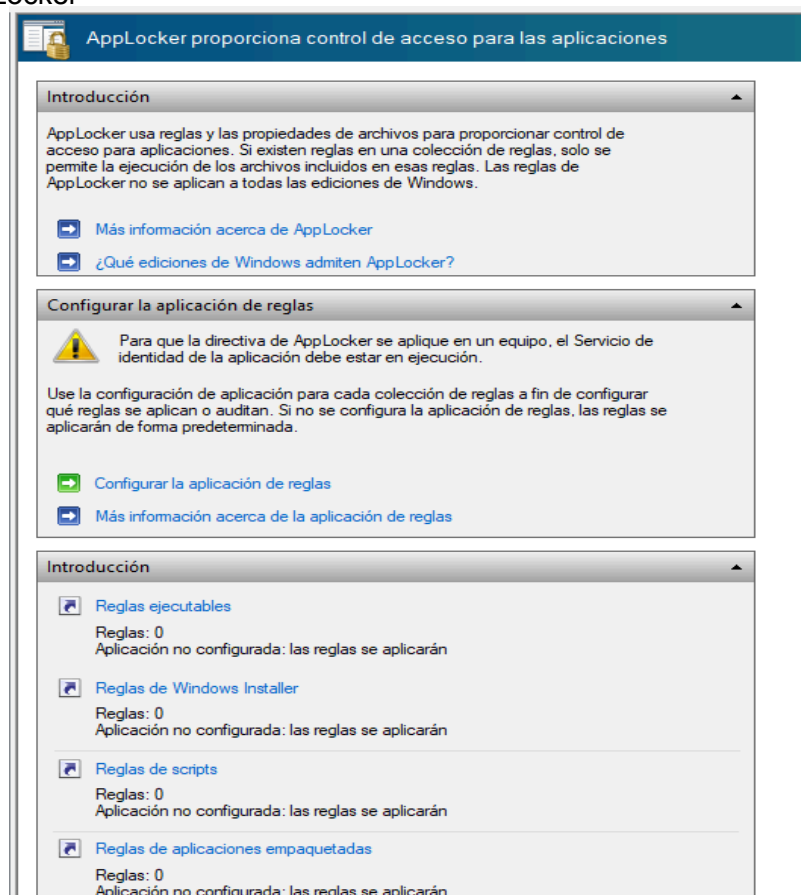
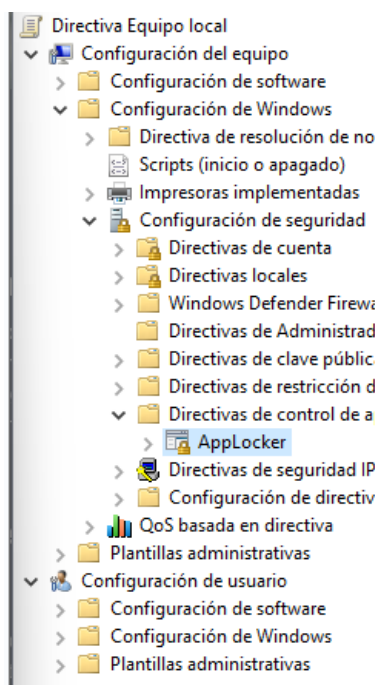


AppLocker se configura mediante políticas de grupo

→ Accederemos a las políticas de grupo (GPO) para llegar a AppLocker

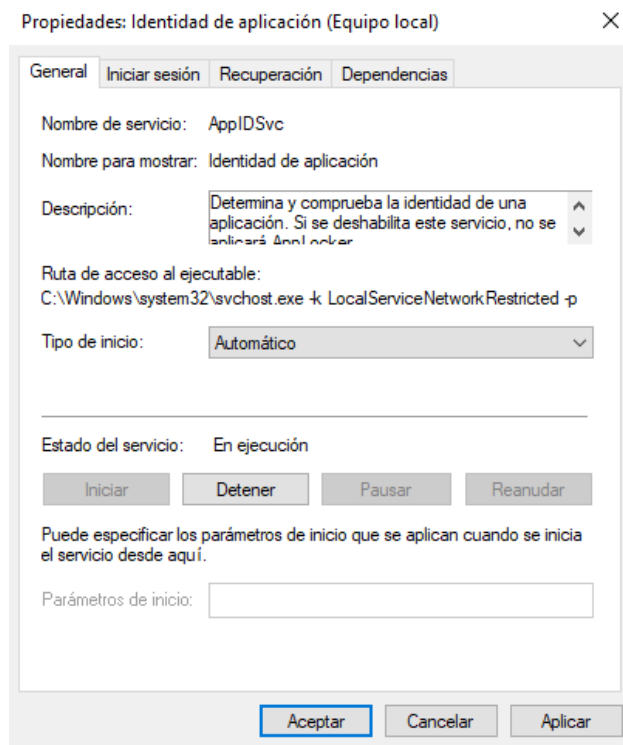
Configuración del equipo → Configuración de Windows → Configuración de seguridad →

Directivas de control de aplicaciones → AppLocker



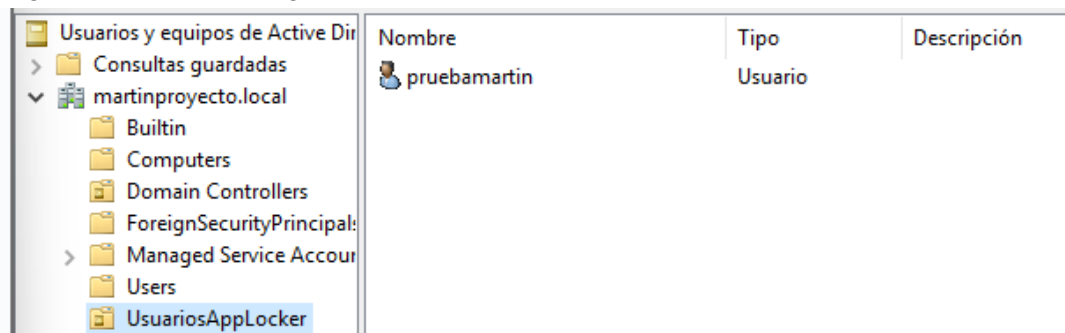
En el cliente de Windows 10 también tendremos que activar el servicio del que hablamos anteriormente, Identidad de Aplicación para poder aplicar AppLocker.

→ Procedemos:



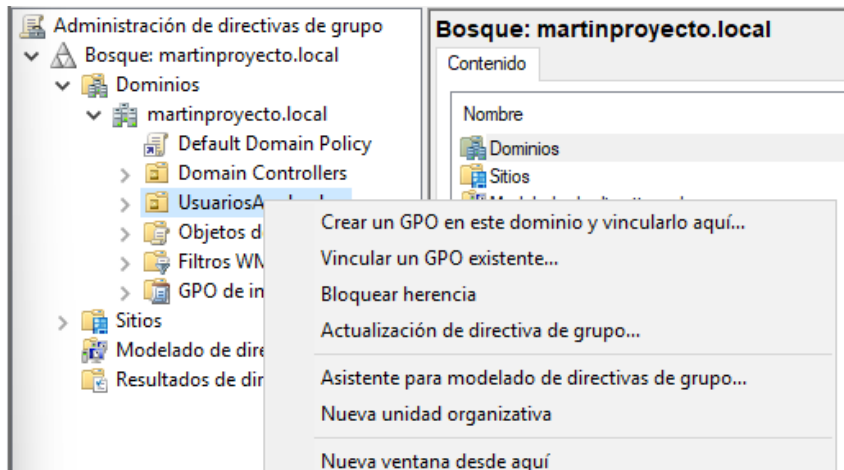
Iniciamos el servicio y lo configuramos en tipo de inicio Automático para que se inicie solo en cada reinicio.

→ Para que las GPO se apliquen a un usuario de dominio, tenemos que crear una Unidad Organizativa (OU) e ingresar al usuario en esa unidad.



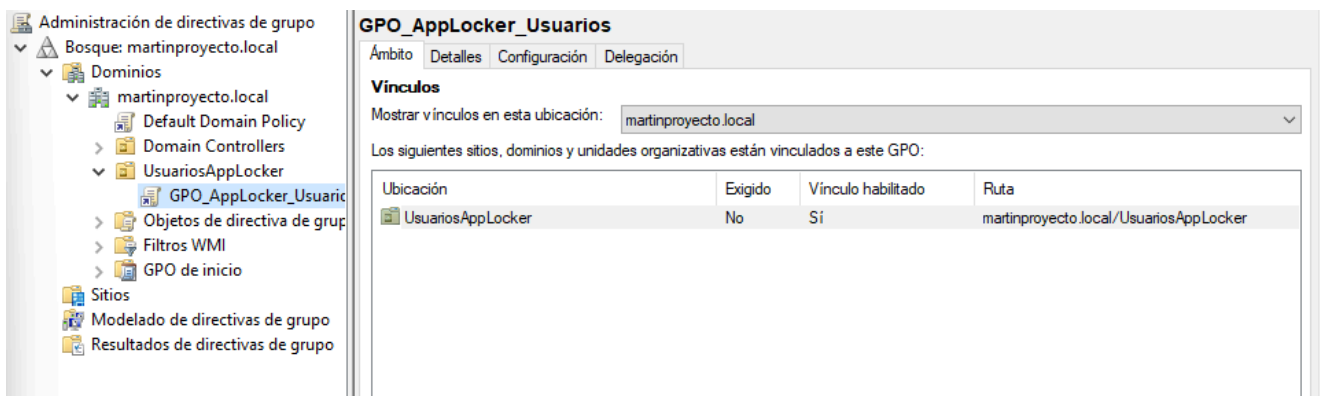
También podríamos ingresar al equipo del cliente en la Unidad Organizativa, por lo que aplicaría a todos los usuarios que inicien sesión en el equipo.

→ Ahora tenemos que crear una GPO nesta Unidad Organizativa:



Accedemos a Administración de directivas de grupo de creamos unha GPO

→ Verificamos:



Ahora comenzamos a creación de reglas.

Para actualizar las reglas no cliente, hay que introducir o comando gpupdate /force y comenzarán a actualizarse as directivas

```
C:\Windows\system32>gpupdate /force
Actualizando directiva...

La actualización de la directiva de equipo se completó correctamente.
Se completó correctamente la Actualización de directiva de usuario.
```

4. CREACIÓN DE REGLAS PARA HARDENING AVANZADO CON APPLOCKER

Comenzamos la creación de reglas:

- 1-Permitir sólo ejecutables firmados por Microsoft en C:\Windows
- 2-Bloquea cualquier .exe fuera de Archivos de programa y Windows
- 3-Bloquear scripts (.ps1, .vbs, .js) fóra de rutas seguras
- 4-Bloquear archivos MSI e instaladores en rutas no autorizadas
- 5-Permitir únicamente scripts concedidos por Microsoft
- 6-Bloquear la ejecución de aplicaciones desde la carpeta del usuario
- 7-Block cmd.exe, powershell.exe y wscript.exe para usuarios estándar

8-Prohibir la ejecución de aplicaciones a un usuario

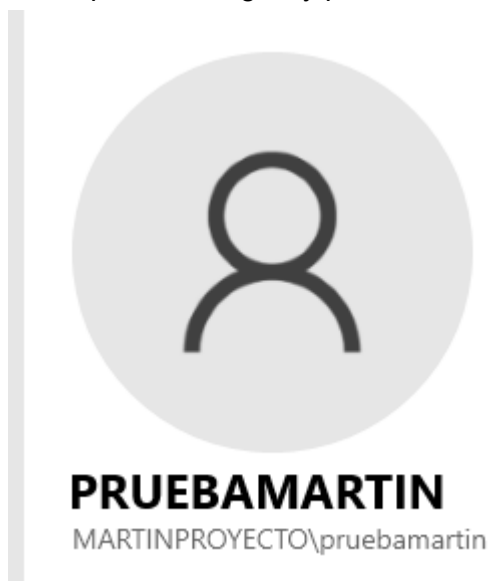
Mediante la implementación de estas 8 reglas de AppLocker, logramos establecer un control muy estricto y efectivo sobre la ejecución de aplicaciones en el sistema operativo Windows. Estas políticas le permiten limitar drásticamente el riesgo de ejecutar software no autorizado, scripts maliciosos e instalaciones no controladas, al tiempo que garantizan la operatividad del sistema para usuarios legítimos.

AppLocker se presenta como una herramienta clave en el bastión de los sistemas Windows, ya que proporciona un marco configurable y detallado para definir quién puede ejecutar qué tipo de aplicaciones y desde dónde. Su correcta aplicación no sólo minimiza la superficie de ataque, sino que también fortalece las políticas de seguridad corporativas al bloquear vectores de malware comunes y técnicas de persistencia.

Las reglas predeterminadas de AppLocker se crearon como punto de partida:

Acción	Usuario	Nombre	Condición
✓ Permitir	Todos	(Regla predeterminada) Todos los archivos de la carpeta Archivos de programa	Ruta de acceso
✓ Permitir	Todos	(Regla predeterminada) Todos los archivos de la carpeta Windows	Ruta de acceso
✓ Permitir	BUILTIN\Ad...	(Regla predeterminada) Todos los archivos	Ruta de acceso

Para aplicar las reglas y pruebas utilizaré el nombre de usuario del Dominio Prueba Martin



1. Permitir únicamente ejecutables firmados por Microsoft en C:\Windows

Con esta regla permitimos la ejecución de ejecutables firmados por Microsoft solo en la ruta del sistema, garantizando la integridad de los binarios críticos.

→ Accedemos a reglas ejecutables e creamos unha nova regra:

Crear Reglas ejecutables

Permisos

Antes de comenzar

Permisos

Condiciones

Editor

Excepciones

Nombre

Seleccione la acción que desee usar y el usuario o grupo al que debe aplicarse esta regla. Una acción de permiso permite que los archivos afectados se ejecuten, mientras que una acción de denegación lo impide.

Acción:

☒ Permitir

☐ Denegar

Usuario o grupo:

MARTINPROYECTO\pruebamartin

Seleccionar...

→ En condición escollemos editor:

Antes de comenzar

Permisos

Condiciones

Editor

Excepciones

Nombre

Seleccione el tipo de condición principal que desee crear.

☒ Editor

Seleccione esta opción si la aplicación para la que desea crear la regla está firmada por el editor de software.

☐ Ruta de acceso

Cree una regla para una ruta de acceso de carpeta o archivo específica. Si selecciona una carpeta, la regla afectará a todos los archivos que contenga.

☐ Hash de archivo

Seleccione esta opción si desea crear una regla para una aplicación no firmada.

→ Ahora escogemos un ejecutable de Microsoft

Archivo de referencia:

C:\Windows\System32\notepad.exe

Examinar...

Cualquier editor

Editor: O=MICROSOFT CORPORATION, L=REDMOND, S=\\

Nombre del producto: MICROSOFT® WINDOWS® OPERATING SYSTEM

Nombre de archivo: *

Versión del archivo: *

Y superior

Dejé el nombre de archivo y la versión en blanco para permitir cualquier archivo de sistema firmado por Microsoft.

No voy a lanzar ninguna excepción ya que solo estoy restringiendo la ejecución a C:\Windows

→ Verificamos que se haya creado correctamente:

Acción	Usuario	Nombre	Condición
✓ Permitir	Todos	(Regla predeterminada) Todos los archivos de la carpeta Ar...	Ruta de a...
✓ Permitir	Todos	(Regla predeterminada) Todos los archivos de la carpeta Wi...	Ruta de a...
✓ Permitir	MARTINPROYECTO\pruebama...	MICROSOFT® WINDOWS® OPERATING SYSTEM, desde O...	Editor
✓ Permitir	BUILTIN\Administradores	(Regla predeterminada) Todos los archivos	Ruta de a...

Así evitamos que ejecutables maliciosos enmascarados se ejecuten desde esa ruta clave.

2. Bloquear cualquier .exe fuera de Archivos de programa y Windows

Denegamos la ejecución de cualquier archivo con la extensión .exe ubicado dentro de cualquier subcarpeta del directorio C:\Users*\AppData\Local* o directamente dentro de la ruta C:\Temp*

→ Creamos la regla:

Seleccione la acción que desee usar y el usuario o grupo al que debe aplicarse esta regla. Una acción de permiso permite que los archivos afectados se ejecuten, mientras que una acción de denegación lo impide.

Acción:

☐ Permitir

☒ Denegar

Usuario o grupo:

MARTINPROYECTO\pruebamartin

Seleccionar...

→ Condición:

Seleccione el tipo de condición principal que desee crear.

☐ Editor

Seleccione esta opción si la aplicación para la que desea crear la regla está firmada por el editor de software.

☒ Ruta de acceso

Cree una regla para una ruta de acceso de carpeta o archivo específica. Si selecciona una carpeta, la regla afectará a todos los archivos que contenga.

☐ Hash de archivo

Seleccione esta opción si desea crear una regla para una aplicación no firmada.

→ En ruta de acceso, poñemos C:\

Seleccione la ruta de acceso de archivo o de carpeta a la que debe afectar esta regla. Si especifica una ruta de acceso de carpeta, esta regla afectará a todos los archivos ubicados en esa ruta de acceso.

Ruta de acceso:

Examinar archivos... Examinar carpetas...

→ Ahora ponemos Archivos de Programa y Rutas de Windows en excepciones:

Condición principal:
%OSDRIVE%*

Agregar excepción:
Ruta de acceso

Excepciones:

Excepción	Tipo
%PROGRAMFILES%*	Ruta de acceso
%WINDIR%\	Ruta de acceso

Agregar... Editar... Quitar...

→ Verificamos:

Acción	Usuario	Nombre	Condición	Excepcio...
✓ Permitir	Todos	(Regla predeterminada) Todos los archivos de la carpeta Ar...	Ruta de a...	
✓ Permitir	Todos	(Regla predeterminada) Todos los archivos de la carpeta Wi...	Ruta de a...	
✓ Permitir	MARTINPROYECTO\pruebama...	MICROSOFT® WINDOWS® OPERATING SYSTEM, desde O...	Editor	
✓ Permitir	BUILTIN\Administradores	(Regla predeterminada) Todos los archivos	Ruta de a...	
✗ Denegar	MARTINPROYECTO\pruebama...	Bloqueo EXE fora de rutas estándar	Ruta de a...	Sí

Así bloqueamos la ejecución de posible malware que se descarga y guarda en rutas comunes del usuario, como el escritorio o las descargas.

3. Bloquear scripts (.ps1, .vbs, .js) fóra de rutas seguras

Esta regla evita que los scripts maliciosos se ejecuten desde carpetas de usuario

→ Para crear esta regla, tenemos que hacerlo en Reglas de Script, antes de nada imos crear las Reglas Predeterminadas de este apartado

→ Creemos la nueva regla: Acción, denegar

Seleccione la acción que desee usar y el usuario o grupo al que debe aplicarse esta regla. Una acción de permiso permite que los archivos afectados se ejecuten, mientras que una acción de denegación lo impide.

Acción:

☐ Permitir

☒ Denegar

Usuario o grupo:

MARTINPROYECTO\pruebamartin

Seleccionar...

→ Condición:

Seleccione el tipo de condición principal que desee crear.

☐ Editor

Seleccione esta opción si la aplicación para la que desea crear la regla está firmada por el editor de software.

☒ Ruta de acceso

Cree una regla para una ruta de acceso de carpeta o archivo específica. Si selecciona una carpeta, la regla afectará a todos los archivos que contenga.

☐ Hash de archivo

Seleccione esta opción si desea crear una regla para una aplicación no firmada.

→ Ruta:

Seleccione la ruta de acceso de archivo o de carpeta a la que debe afectar esta regla. Si especifica una ruta de acceso de carpeta, esta regla afectará a todos los archivos ubicados en esa ruta de acceso.

Ruta de acceso:

%OSDRIVE%\Users*

Examinar archivos...

Examinar carpetas...

→ En este caso no crearemos ninguna excepción.

→ Verificamos:

Acción	Usuario	Nombre	Condición	Excepcio...
✓ Permitir	MARTINPROYECTO\pruebamartin	Permitir scripts solo firmados por Micro...	Editor	
✓ Permitir	Todos	(Regla predeterminada) Todos los script...	Ruta de a...	
✓ Permitir	Todos	(Regla predeterminada) Todos los script...	Ruta de a...	
✓ Permitir	BUILTIN\Administradores	(Regla predeterminada) Todos los scripts	Ruta de a...	
✗ Denegar	MARTINPROYECTO\pruebamartin	Bloqueo de scripts en perfiles de usuario	Ruta de a...	

De esta manera nos defendemos contra ataques de scripts (por ejemplo, PowerShell malicioso) ejecutados desde descargas, escritorio, etc.

4. Bloquear archivos MSI e instaladores en rutas no autorizadas

Con esta regla de AppLocker evitamos que los archivos .msi (instaladores de Windows) se ejecuten dentro de cualquier carpeta de usuario en %OSDRIVE%\Users*

→ Creamos la regla, en este caso será en Windows Installer por lo que crearemos las reglas por defecto

→ En acción, denegamos:

Seleccione la acción que desee usar y el usuario o grupo al que debe aplicarse esta regla. Una acción de permiso permite que los archivos afectados se ejecuten, mientras que una acción de denegación lo impide.

Acción:

☐ Permitir

☒ Denegar

Usuario o grupo:

MARTINPROYECTO\pruebamartin Seleccionar...

→ En condición:

Seleccione el tipo de condición principal que desee crear.

☐ Editor

Seleccione esta opción si la aplicación para la que desea crear la regla está firmada por el editor de software.

☒ Ruta de acceso

Cree una regla para una ruta de acceso de carpeta o archivo específica. Si selecciona una carpeta, la regla afectará a todos los archivos que contenga.

☐ Hash de archivo

Seleccione esta opción si desea crear una regla para una aplicación no firmada.

→ Ruta: Carpeta de usuarios

Ruta de acceso:

→ No creamos ninguna excepción

→ Verificamos:

Acción	Usuario	Nombre	Condición	Excepcio...
Permitir	Todos	(Regla predeterminada) Todos los archivos d...	Editor	
Permitir	Todos	(Regla predeterminada) Todos los archivos d...	Ruta de acceso	
Permitir	BUILTIN\Ad...	(Regla predeterminada) Todos los archivos d...	Ruta de acceso	
Denegar	MARTINPRO...	Bloqueo de instaladores a usuarios	Ruta de acceso	

Así reducimos o riesgo de instalación de software non controlado (e posiblemente peligroso)

5. Permitir únicamente scripts otorgados por Microsoft

Esta regla permite a ejecución solo de scripts de confianza concedidos/permitidos digitalmente por Microsoft.

→ Regresamos a las Reglas del Script y creamos una nueva:

Permisos

Antes de comenzar

Permisos

Condiciones

Editor

Excepciones

Nombre

Seleccione la acción que desee usar y el usuario o grupo al que debe aplicarse esta regla. Una acción de permiso permite que los archivos afectados se ejecuten, mientras que una acción de denegación lo impide.

Acción:

☒ Permitir

☐ Denegar

Usuario o grupo:

MARTINPROYECTO\pruebamartin

Seleccionar...

→ Condición:

Seleccione el tipo de condición principal que desee crear.

☒ Editor

Seleccione esta opción si la aplicación para la que desea crear la regla está firmada por el editor de software.

☐ Ruta de acceso

Cree una regla para una ruta de acceso de carpeta o archivo específica. Si selecciona una carpeta, la regla afectará a todos los archivos que contenga.

☐ Hash de archivo

Seleccione esta opción si desea crear una regla para una aplicación no firmada.

→ En el Editor, seleccionemos un ejecutable de Microsoft, como notepad.exe. Luego seleccionamos el nivel de editor (Microsoft Corporation) y dejamos el resto en * para cualquier archivo.

Archivo de referencia:

C:\Windows\System32\notepad.exe Examinar...

- Cualquier editor

- Editor: O=MICROSOFT CORPORATION, L=REDMOND, S=\

 - Nombre del producto: MICROSOFT® WINDOWS® OPERATING SYSTEM

- Nombre de archivo: *

- Versión del archivo: * Y superior ▼

→ Non añadimos ninguna excepción

→ Verificamos:

Acción	Usuario	Nombre	Condición	Excepcio...
 Permitir	BUILTIN\Administradores	(Regla predeterminada) Todos los scripts	Ruta de a...	
 Permitir	Todos	(Regla predeterminada) Todos los script...	Ruta de a...	
 Permitir	Todos	(Regla predeterminada) Todos los script...	Ruta de a...	
 Denegar	MARTINPROYECTO\pruebamartin	Bloqueo de scripts en perfiles de usuario	Ruta de a...	
 Permitir	MARTINPROYECTO\pruebamartin	Permitir scripts solo firmados por Micro...	Editor	

Así solo se permiten scripts oficiales e firmados por Microsoft, evitando PowerShells maliciosos

6. Bloquear aplicaciones en ejecución desde la carpeta del usuario

Bloquea cualquier archivo .exe que se copie o descargue en el escritorio, descargas, documentos u otras rutas de perfil de usuario, que es donde normalmente se almacenan las aplicaciones portátiles o maliciosas.

→ En reglas ejecutables, introducimos a acción:

Seleccione la acción que desee usar y el usuario o grupo al que debe aplicarse esta regla. Una acción de permiso permite que los archivos afectados se ejecuten, mientras que una acción de denegación lo impide.

Acción:

☐ Permitir

☒ Denegar

Usuario o grupo:

MARTINPROYECTO\pruebamartin Seleccionar...

→ En condiciones, ruta de acceso

→ Ruta de acceso:

Ruta de acceso:

→ En excepciones podríamos agregar algunas si es necesario para ese perfil, como aplicaciones Edge o algo como \Usuarios\NombreDeUsuario\Microsoft\Edge\Aplicación
En mi caso no añadiré nada.

→ Verificamos:

Acción	Usuario	Nombre	Condición	Excepcio...
✓ Permitir	BUILTIN\Administradores	(Regla predeterminada) Todos los archivos	Ruta de a...	
✓ Permitir	Todos	(Regla predeterminada) Todos los archivos de la carpeta Ar...	Ruta de a...	
✓ Permitir	Todos	(Regla predeterminada) Todos los archivos de la carpeta Wi...	Ruta de a...	
✓ Permitir	MARTINPROYECTO\pruebama...	MICROSOFT® WINDOWS® OPERATING SYSTEM, desde O...	Editor	
✗ Denegar	MARTINPROYECTO\pruebama...	Bloquear ejecutables nas rutas do usuario	Ruta de a...	
✗ Denegar	MARTINPROYECTO\pruebama...	Bloqueo EXE fora de rutas estándar	Ruta de a...	Sí

Esto evita que los usuarios ejecuten software portátil descargado o copiado localmente en sus computadoras personales, una de las rutas de entrada más comunes para malware y herramientas maliciosas.

7. Bloquear cmd.exe, powershell.exe y wscript.exe para usuarios estándar

Evitamos que usuarios sin privilegios administrativos usen herramientas de sistema peligrosas.

→ Seguimos en reglas ejecutables. Applocker non permite crear una regla con tres rutas de acceso diferentes, por lo tanto, tenemos que crear tres reglas, una para cada ejecutable

→ Denegamos permisos y los aplicamos al usuario *prueba martin*

→ Condiciones, ruta de acceso

Ruta de acceso:

Ruta de acceso:

Ruta de acceso:

Examinar archivos...
Examinar carpetas...

Ruta de acceso:

Examinar archivos...
Examinar carpetas...

→ Así xa estarían bloqueados los 4 ejecutables dos programas peligrosos. Comprobamos:

	Denegar	MARTINPROYECTO\pruebama...	Restringir cmd.exe a usuarios	Ruta de a...
	Denegar	MARTINPROYECTO\pruebama...	Restringir powershell.exe a usuarios	Ruta de a...
	Denegar	MARTINPROYECTO\pruebama...	Restringir powershell_ise.exe a usuarios	Ruta de a...
	Denegar	MARTINPROYECTO\pruebama...	Restringir wscript.exe a usuarios	Ruta de a...

Con esto previmos que los usuarios usen PowerShell o CMD para ejecutar comandos maliciosos.

8. Prohibir ejecución de aplicaciones a un usuario

Esta regla prohíbe el uso de aplicaciones no deseadas para reducir la posibilidad de peligro. En mi caso vamos a prohibir Internet Explorer.

→ Creamos la regla nova, denegamos permisos e aplicamos la ó usuario

Seleccione la ruta de acceso de archivo o de carpeta a la que debe afectar esta regla. Si especifica una ruta de acceso de carpeta, esta regla afectará a todos los archivos ubicados en esa ruta de acceso.

Ruta de acceso:

Examinar archivos...
Examinar carpetas...

También haré lo mismo, por ejemplo, para el Calendario o la Calculadora.

→ Creamos la regla y comprobamos que se creó:

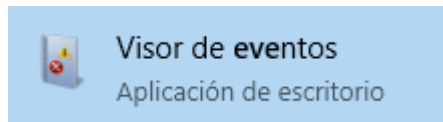
	Denegar	MARTINPROYECTO\pruebama...	Prohibir uso de iexplore.exe	Ruta de a...
--	---------	----------------------------	------------------------------	--------------

De esta forma garantizamos que la aplicación no sea utilizada, manteniendo la seguridad.

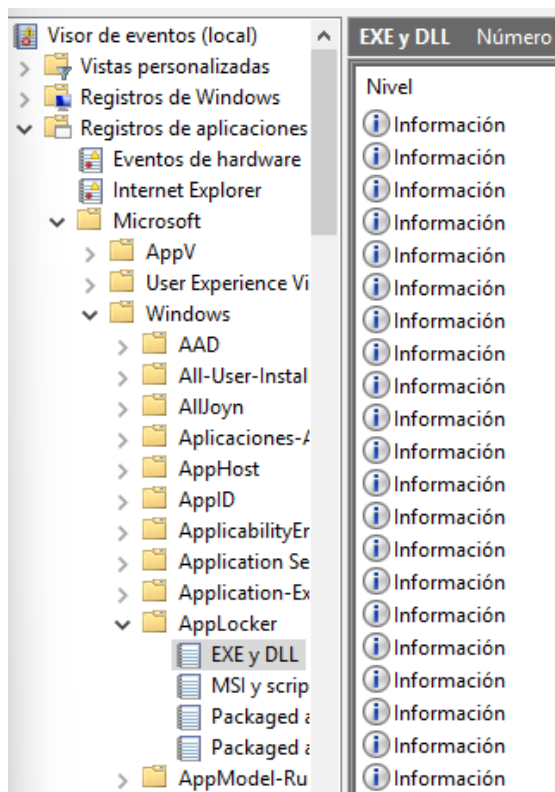
5. Visor de eventos de Applocker

Permite auditar la ejecución de aplicaciones, verificar qué reglas se están aplicando y detectar intentos de ejecución bloqueados.

Para acceder tenemos que abrir o visor de eventos

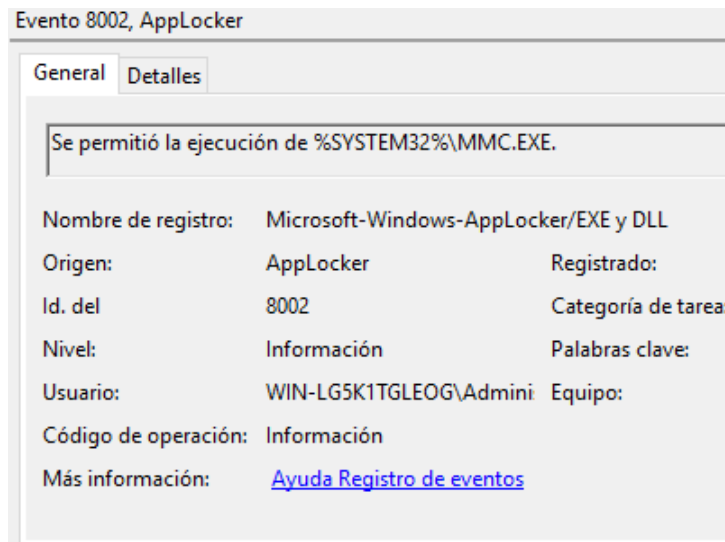


Una vez dentro, Registros de aplicaciones y servicios > Microsoft > Windows > AppLocker > EXE and DLL

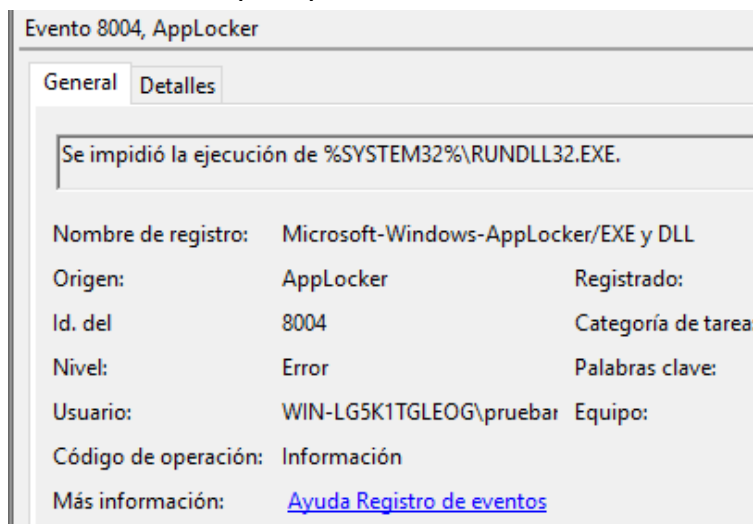


Aquí podemos ver todos los eventos de información y también los intentos de acceso.

Eventos de información (8002):



eventos de error(8004):



Para que se utiliza:

- Verificar si las reglas funcionan correctamente.
- Identificar intentos de ejecución bloqueados por parte de los usuarios.
- Detectar posibles intentos de ejecución de malware o software no autorizado.
- Ayudar en el ajuste de las políticas sin romper el sistema.

6. PROBA REGLAS DE APPLOCKER

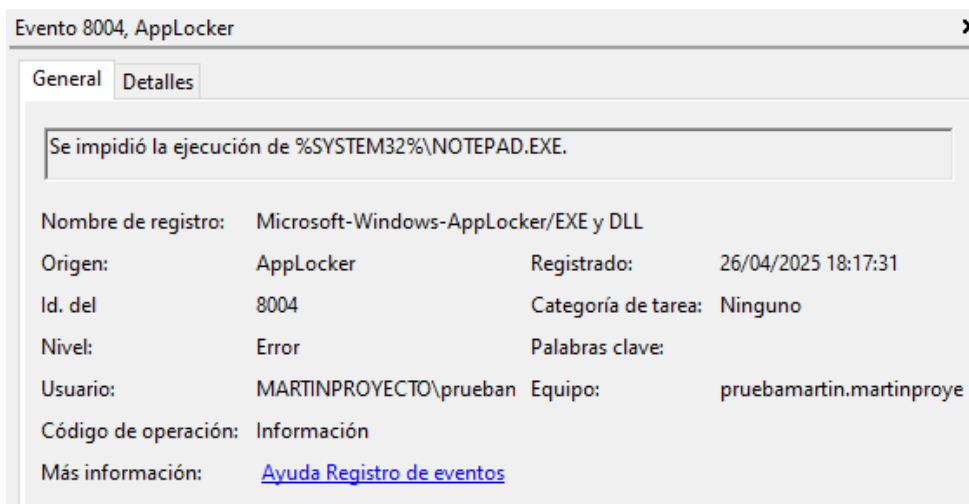
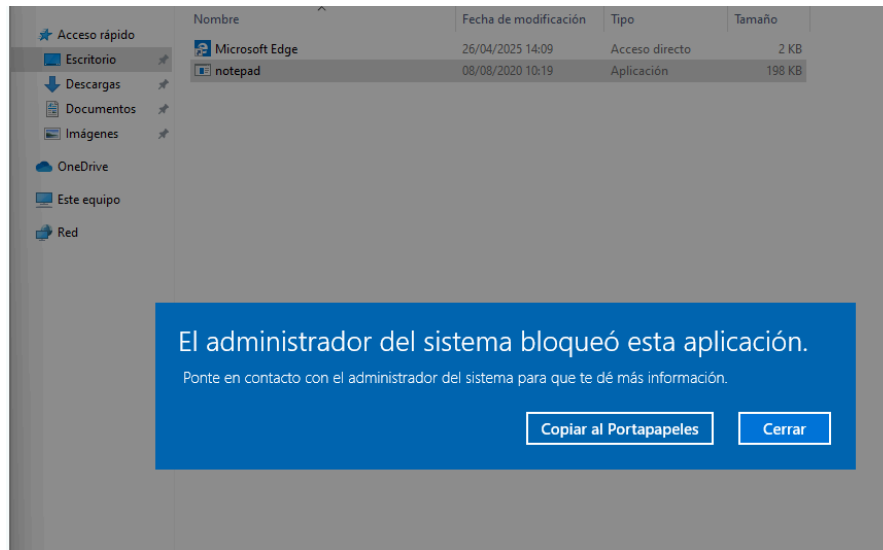
Comenzamos ca probadas reglas:

1-Permitir sólo ejecutables firmados por Microsoft en C:\Windows

[video solo firmados microsoft](#)

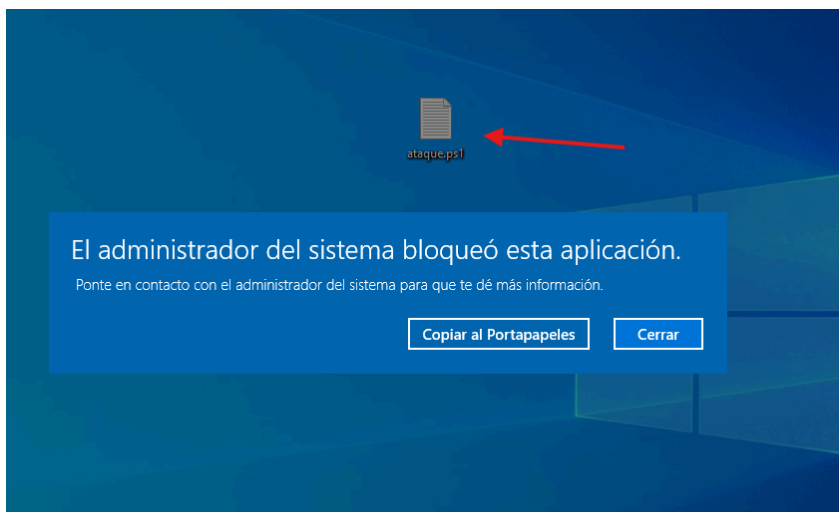
2-Bloquea cualquier .exe fuera de Archivos de programa y Windows

[video .exe fuera de Archivos de programa y Windows](#)



3-Bloquear scripts (.ps1, .vbs, .js) fóra de rutas seguras

[scripts de bloqueo de vídeo](#)



4-Bloquear archivos MSI e instaladores en rutas no autorizadas

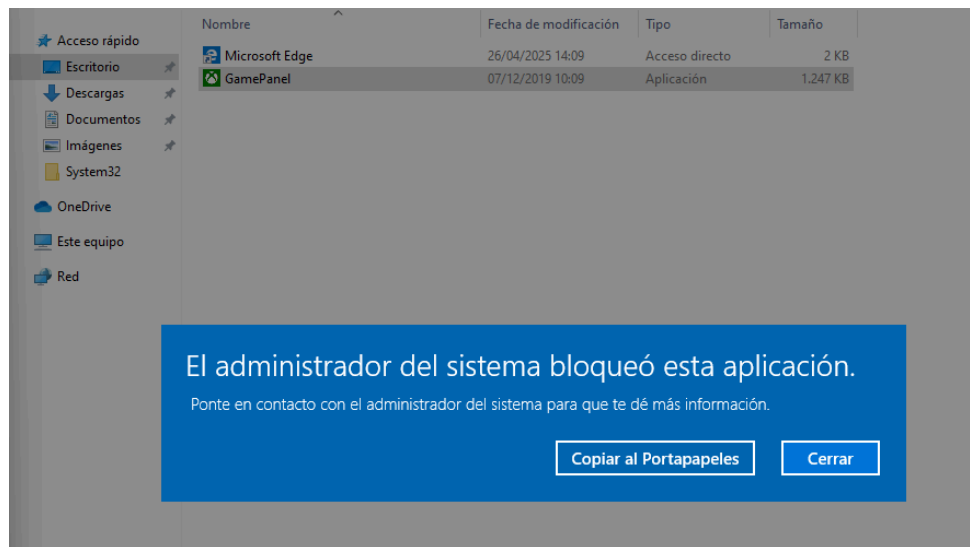
[video Bloqueo MSI](#)

5-Permitir únicamente scripts concedidos por Microsoft

[guiones de vídeo otorgados por Microsoft](#)

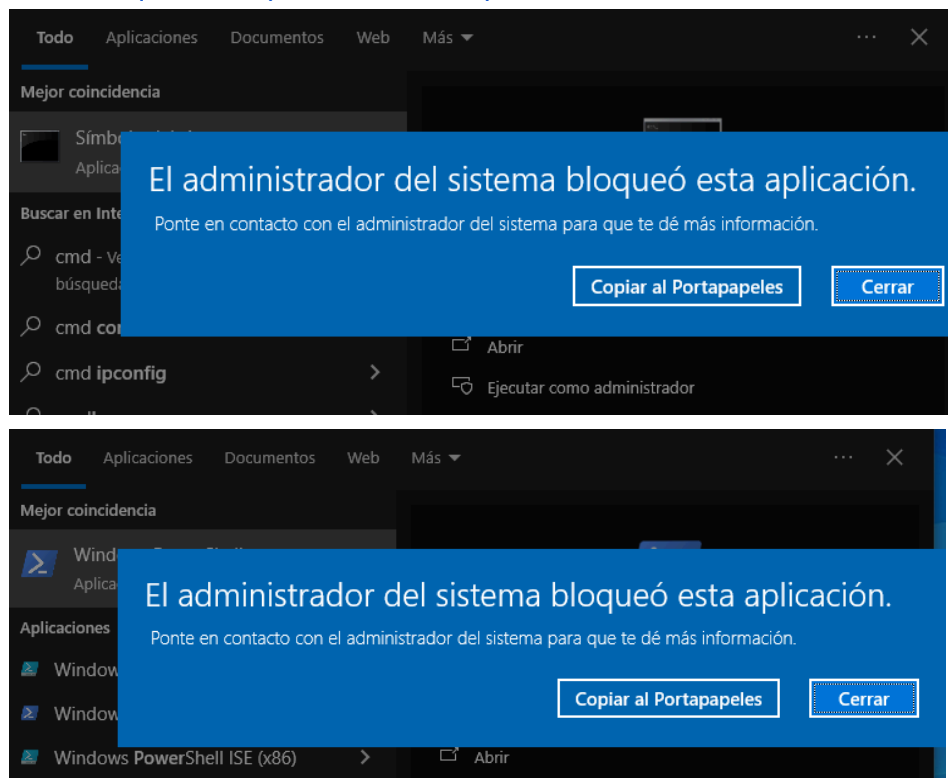
6-Bloquear la ejecución de aplicaciones desde la carpeta del usuario

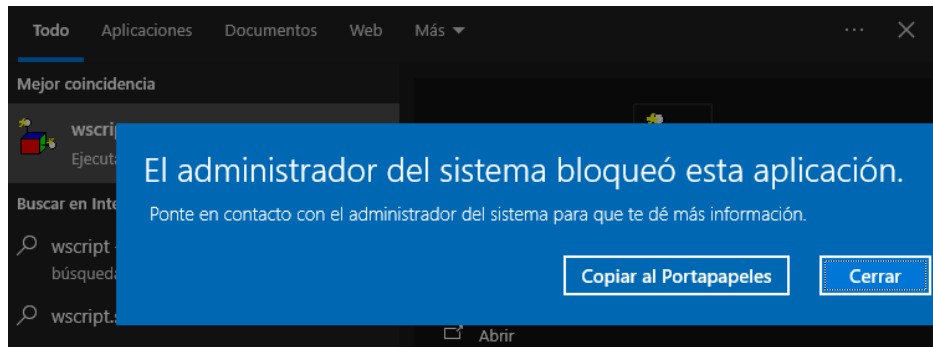
[video aplicación desde a ruta do usuario](#)



7-Block cmd.exe, powershell.exe y wscript.exe para usuarios

[Vídeo bloqueo cmd-powershell-wscript](#)





8-Prohibir la ejecución de aplicaciones a un usuario

[vídeo prohibir aplicaciones](#)

6. BIBLIOGRAFÍA / WEBGRAFÍA

Aprendizaje (Principales) obtenido en Bastionado de Redes y Sistemas y también del resto de módulos del Máster de Ciberseguridad

<https://www.elladodelmal.com/2020/01/servicios-ssh-tips-de-auditoria-y.html>

https://docs.redhat.com/es/documentation/red_hat_enterprise_linux/8/html-single/using_selinux/index

<https://www.tarlogic.com/es/blog/selinux-para-proteger-una-web/>

<https://puerto53.wordpress.com/2021/08/08/guia-basica-de-selinux/>

<https://books.spartan-cybersec.com/cpad/introduccion-a-la-evasion-de-defensas/introduccion-a-applocker>

<https://www.asnet.es/utilizacion-applocker-bloquear-aplicaciones-sobre-windows-10-servidor-de-rds/>

[https://github.com/DarbyShin/Windows-10-IoT-Enterprise/blob/master/Guía paso a paso - Guía de bloqueo de aplicaciones de Windows 10.pdf](https://github.com/DarbyShin/Windows-10-IoT-Enterprise/blob/master/Guía%20paso%20a%20paso%20-%20Guía%20de%20bloqueo%20de%20aplicaciones%20de%20Windows%2010.pdf)

<https://learn.microsoft.com/es-es/windows/security/application-security/application-control/app-control-for-business/applocker/working-with-applocker-rules>